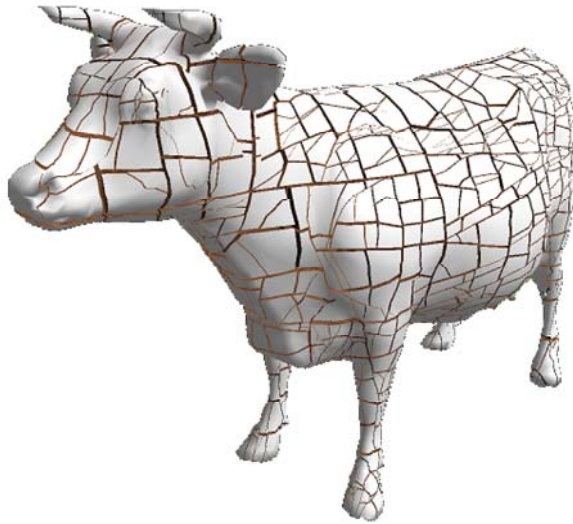


Destruction Authoring with PhysXLab

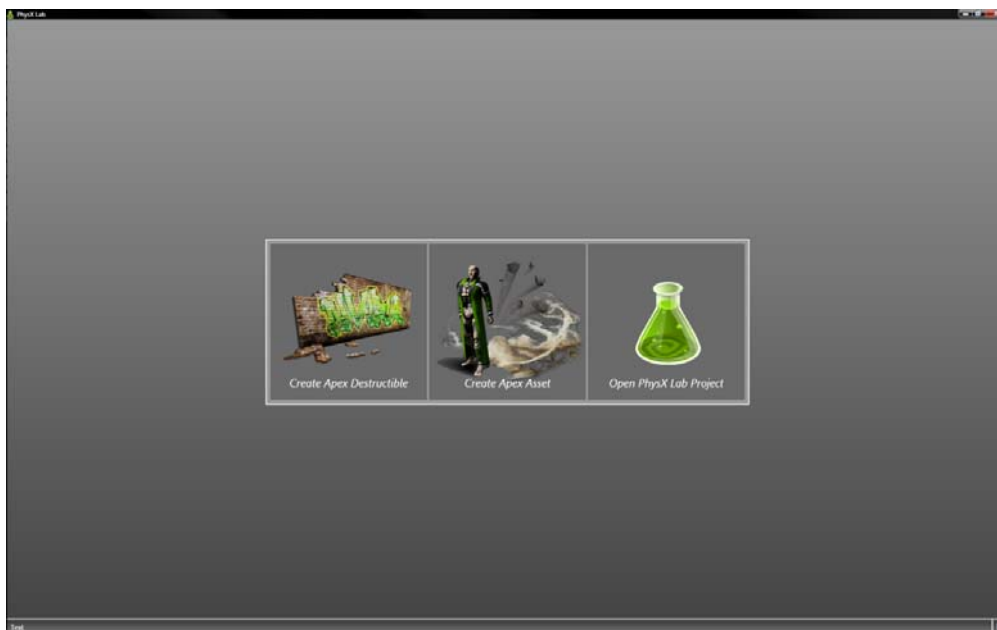


Destruction authoring is a component of PhysXLab, a standalone GUI authoring tool for APEX. Using PhysXLab, you may take user-provided artwork (meshes) and split them into broken fragments or "chunks." The result can be used to generate an APEX Destructible Asset (NxDestructibleAsset), which can be streamed into the standard APEX asset (.apx and .apb) file formats. This not only contains a hierarchy of fractured meshes, but also collision information and physical material parameters, as well as support information for each chunk. Besides exporting APEX file formats, you can also save various aspects of your workspace. The entire workspace can be saved into a project (.npproj) file, or you can just save the current mesh into an .afm ("apex fractured mesh") file. You can also save the current mesh's material library in an .aml ("apex material library") file. This can be used as a material source when choosing a fractured mesh's interior material. Lastly, an important subset of the project is the "fracture template." These parameters control the splitting of the mesh into chunks, and may be tuned to give various effects that emulate the fracturing of wood, concrete, glass, etc. The user may save off the fracture template into an .nppf ("preset fracture") file.

Getting started

Note This section will quickly run you through the controls you need to use to perform some basic fracturing. It is not meant to be a full description of the functionality of PhysXLab destruction. The next section will cover all controls in detail.

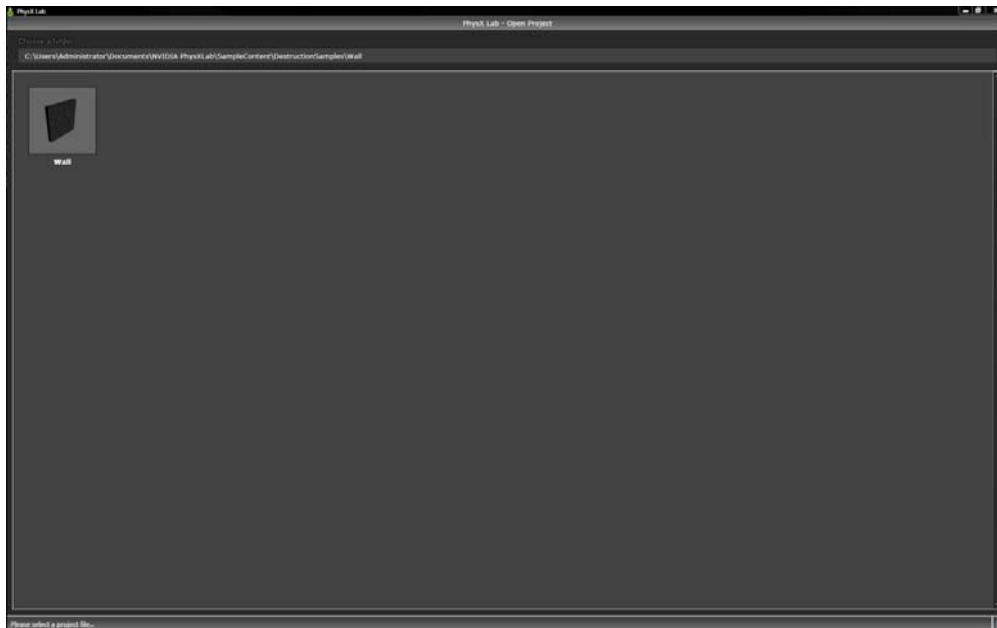
Run PhysXLab.exe. You should see this application



Select "Create Apex Destructible" for Destruction or "Open PhysX Lab Project" to open an existing project. "Create Apex Asset" starts a generic editor which is beyond the scope of this tutorial.

Loading a Pre-existing Project

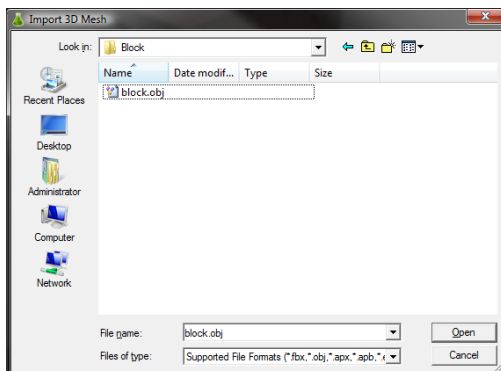
You can open an existing project by selecting "Open PhysX Lab Project" from the screen above or select "Open" in the "File" menu after you've clicked on "Create Apex Destructible". You should see this open project screen



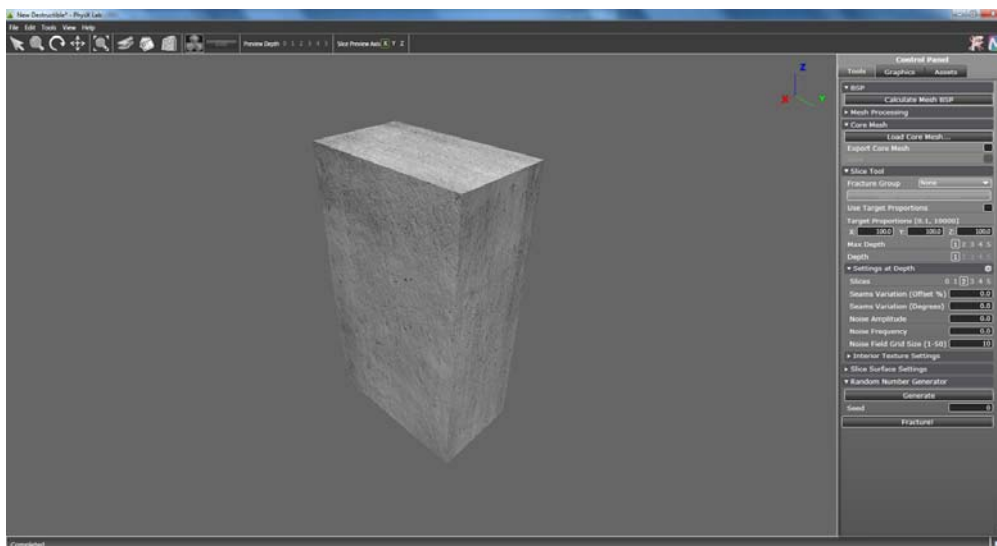
The project folder must first be selected, by clicking on the folder path under "Choose Project Folder." When you do this, a folder selection dialog will appear, allowing you to choose the folder containing your project file(s). Once this is selected, icons for all PhysXLab projects will appear in the large pane in this introduction screen. Each will contain a small preview image and the name of the project. Click on the one you wish to load.

Starting a New Project

Once you clicked "Create Apex Destructible", a new destructible project is created. Click on "Import 3D mesh". A file dialog will appear allowing you to choose a mesh to import. This mesh is the starting point for a destructible asset. Several file formats are supported: the Autodesk .fbx, the Alias .obj, APEX XML .apx, APEX binary .apb, Easy Mesh .ezm file formats. With .obj files, if you have materials applied to your mesh, make sure you also have the material library (.mtl file) in the same directory as the .obj file. For both file formats, you'll also need to have the materials' texture map files in the same directory as the import mesh file. PhysXLab will load .dds, .bmp, and .tga textures.



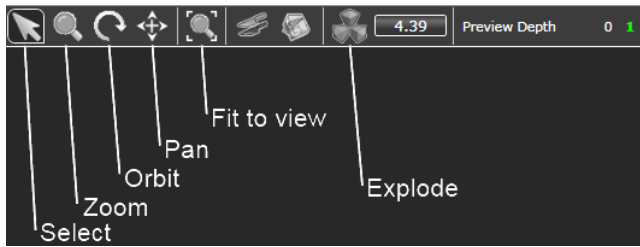
After importing the block.obj file you should see



If your mesh is incorrectly oriented, you might need to change your 'up' axis. Do this under the Views menu (the default is Z-up).

Manipulating the View

The toolbar at the top of the main panel allows you to select the manipulation mode.



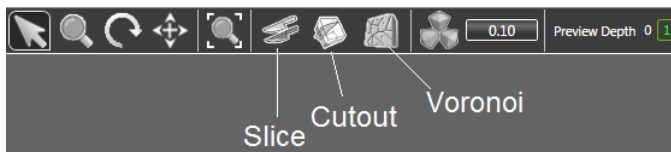
Manipulation Modes

Mode	Description
Select (arrow)	Left-click disables all camera manipulations and allows selection of chunk.
Zoom (magnifying lens)	Left-click and drag will move the mesh towards or away from the camera.
Pan (arrows and hand)	Left-click and drag will translate the mesh in a direction perpendicular to the line of sight.
Orbit (arrows on circles)	Left-click and drag will rotate the mesh.
Explode Tool (expanding shapes)	Left-click and drag will change the distance between chunks at the currently displayed hierarchy depth. Clicking on a chunk with the center crosshair will toggle selection of the chunk. (Selected chunks are highlighted in red.)
Fit to view (box)	Automatically pans and zooms to fit the mesh within the viewport.

Fracturing the Mesh

There are three fracture modes, or "patterns." The first is called "Slicing." With Slicing, the mesh is sliced automatically along various axes. It is recursive, so that each piece obtained by slicing can be sliced again. The user can control various parameters such as the number of slices along each axis and random variations in the slicing surface. These parameters are set for each level (depth) in the fracture hierarchy. The second pattern is called "Cut-out." With cut-out fracturing the user supplies a fracture map in the form of a bitmap file, with fracture lines drawn on it. The mesh is then fractured along those lines to a depth chosen by the user. This can be done from six different directions. The third pattern is called "Voronoi." By this manner a 3-D Voronoi partition will be calculated first, and then the partition is applied to the mesh, thus divided into several polyhedrons. What the user should provide is a number that specifies how many parts will be resulted.

The toolbar at the top of the main panel allows you to select the fracture mode.



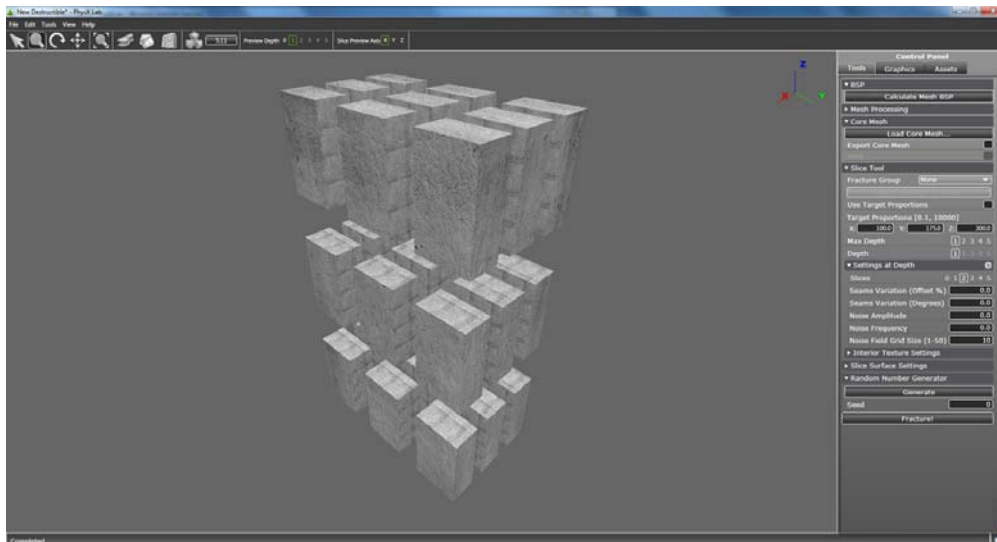
Fracture Modes

Mode	Description
Slicing (knives)	Selects slice fracturing. Slicing parameters will appear in the tool panel on the right.
Cutout (block with chipped face)	Selects cut-out fracturing. Cut-out parameters will appear in the tool panel on the right.
Voronoi (Voronoi partitioned face)	Selects voronoi fracturing. Voronoi parameters will appear in the tool panel on the right.

We will first walk through the default slicing pattern.

Press the "Fracture!" button at the bottom of the tools panel on the right. (Alternatively, you may click on the "F" icon in the lower-right part of the main view panel.) This starts the splitting process. A progress bar will show the splitting progress at the bottom of the application window. After splitting has completed, the application generates an `NxApeRenderMeshAsset` object for the split mesh. This is used (a) to display the split mesh in the window and (b) when you choose to create a destructible asset. All of this may happen very quickly for a simple mesh.

After fracturing has completed, the smallest (hierarchically deepest) chunks will be shown, slightly separated. The fracture mode will be set to the explode tool (see above), so that left-click-drag will change the separation between chunks.



To view chunks at hierarchy depths other than the deepest, use the "Depth (Preview)" control in the Tools panel on the left. The default slicing parameters only slice to a depth of one, so for these settings your only choices are depth = 0 and depth = 1. Depth = 0 is always the original, unfractured mesh.

Noisy Fracture

Default fracturing uses smooth splitting planes. If you want the fracture to appear rough or uneven, you may add noise to the fracture surface by increasing the Variation and Noise parameters in the Tools panel. For slice mode facturing, you may easily introduce noise by setting the Seams Variation, Seams Offset, and Noise fields in the slice tools panel (on the right).

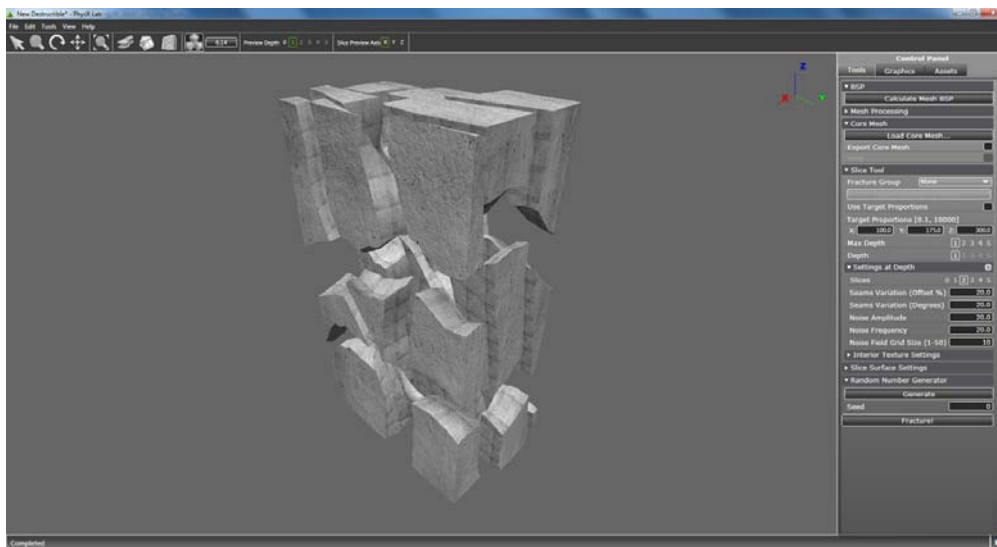
Seams Variation (Offset %) is a random offset for the slicing surfaces, so they won't be evenly spaced. The number of slices along each axis is given by the Slices control. If you imagine the mesh being divided up into N equally spaced sections along an axis, then the Offset controls where the slicing plane displacements may lie within those regions. Offset = 0 means the plane displacements will always be centered in the regions. Offset = 100% means the plane displacements may lie anywhere within those regions, at a randomly chosen location.

Seams Variation (Degrees) is a random angle for the slicing surfaces, so they won't all be axis-aligned. The value you choose is the maximum angle (in degrees) that the slicing plane normals may deviate from axis-alignment.

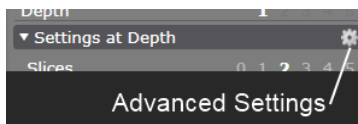
The Seams Variation controls have been described in terms of slicing planes. If you choose to use a noisy splitting surface instead of a plane, then the same notion still applies, but to the surface's "base" plane. Surface noise is then applied to the base plane, with parameters set through the Noise control.

Noise controls both the amplitude and frequency of random variations applied to the splitting surfaces. The value is a percentage - in amplitude this is a percentage of the mesh size, and in frequency this is a percentage of the reciprocal of the grid spacing. (Noise grid spacing control will be discussed later.) Be careful adding noise. It makes much more complex surfaces, which tend to take much more memory to store. It also can take a long time to calculate many recursive splits with high noise along all three axes.

As an example, if you enter 20 for Seams Variation (both Offset % and Degrees) and also in Noise, then hit "Fracture!," you will get the following fractured mesh.



More detailed control over the slicing parameters along each axis is available, if you click in the "Sprocket" button on the right side of the "Settings at Depth" expander.



This toggles the Slice Tool controls to "Advanced" mode and allows you to set all values on a per-axis basis, as well as set the noise amplitude and frequency separately. In this mode you can also set the minimum number of gridpoints across the mesh for splitting along the three axes.

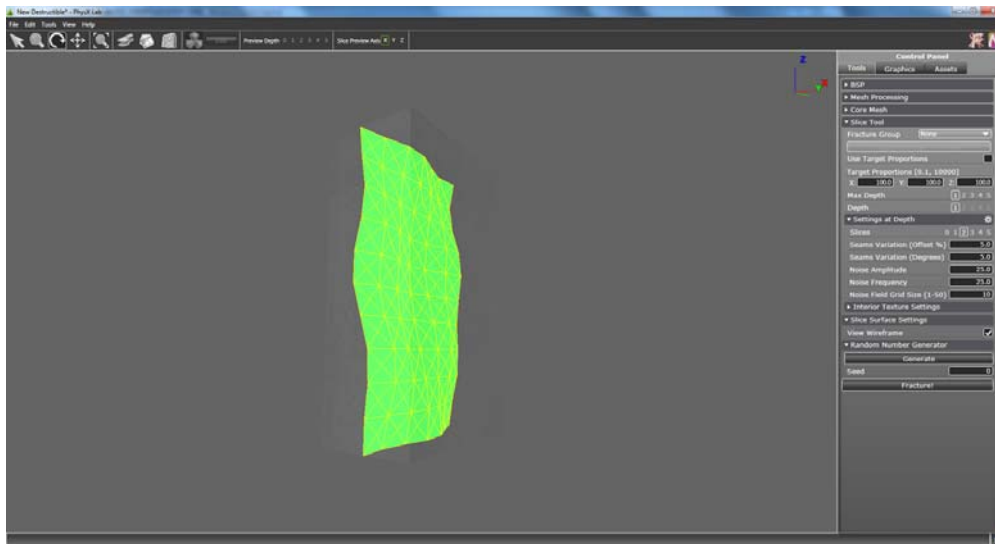
Previewing Typical Slice Surface

When you specify the noise control parameters for slicing, you might want to imagine what the slice effect would be. That is, what the shapes would look like for the sliced chunks.

Choose the menu item "Display Typical Slice Surface" under View. A curved surface will be embedded in the mesh, displaying how the noise will work. At the same time the mesh will become transparent, thus visualizing the noise surface.

There also is an option for you to view the noise surface covered with a wireframe, which could make the shape more visible. This could help much especially under Cut-out Fracturing Mode, about which will be talked soon. To show the wireframe, just check the **View Wireframe** checkbox under the expander of **Slice Surface Settings**.

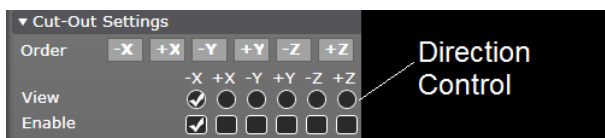
See the example of such a surface, covered with the wireframe.



You can view the surface from different axis directions. In Slice mode, you can change the direction by clicking on the Direction Control shown in the following image



while change in Cut-out mode by clicking the Direction Control shown in this one



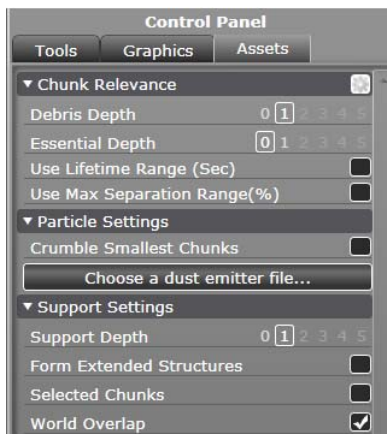
Previewing Behavior in the Playground

Once you have split the mesh, a destructible asset is created. You may preview the runtime behavior of the destructible asset in the Playground. But first, you probably will want to set some parameters in the asset to control its behavior.

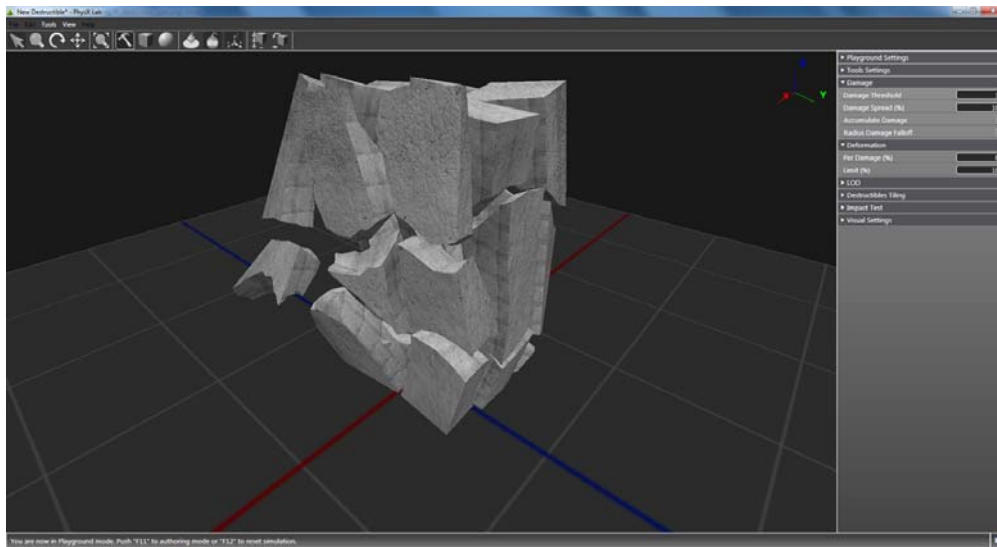
Click on the "Assets" icon in the Control Panel (in the upper right). One thing you'll almost always want to choose is a "support depth" for your asset. This is something that is hard for PhysXLab to choose automatically. The best value usually represents a tradeoff between behavior and performance. Put simply, the support depth is the maximum hierarchy level at which chunks know how to "stick together." Any chunks deeper than the support depth automatically become free once fracturing occurs down to a level at which they're exposed. Ideally, you'd probably want the support depth to be as deep as possible, that is the maximum chunk depth. But for deep enough hierarchies, say down to a depth of three or greater, the number of chunks becomes large enough that maintaining the corresponding support graph can become cost prohibitive.

In the examples so far, we've only used fracturing to depth = 1, so go ahead and choose Support Depth = 1 in the Support Settings section. (See the image above.)

Another thing you often want to do, for a destructible that is supposed to be part of a fixed structure (such as a wall or pillar, e.g.), is to choose a mode of "environmental support." That is, APEX destructibles can mimic being attached to the world. When that attachment is severed, the rest of the destructible can break free. An easy way to do this is to choose "World Overlap." When this is chosen, any chunks at the Support Depth which overlap (or are adjacent to) world geometry will be "environmentally supported." In the Playground there is only one piece of world geometry, a ground plane, but it serves nicely for this example. **Note**, there is a second way of defining environmentally supported chunks, by choosing the chunks which are to be environmentally supported and choosing "Selected Chunks." The interaction between "Selected Chunks" and "World Overlap" when both are selected is described below in the Support Settings section.



So, selecting World Overlap (see the image above), we are ready to enter the Playground. To do this, simply hit F12 (or choose Playground in the Tools menu.) You will see the destructible on a ground plane, and your cursor will change into a hammer. Clicking on the destructible anywhere will apply damage at that point. You should have something like what is seen in the image below.



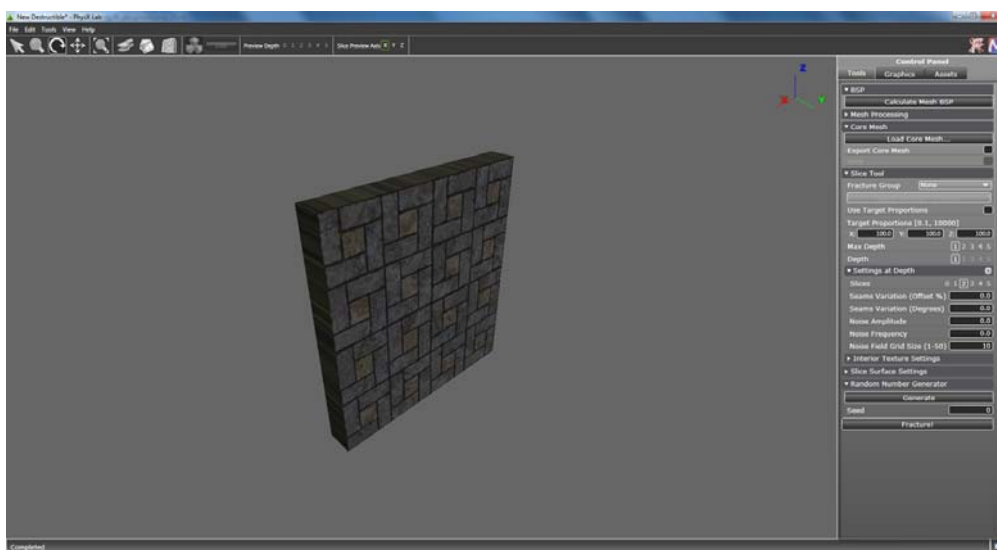
To exit Playground mode, hit F11.

Cut-out Fracturing

Another mode of fracture, cut-out, allows an artist to define fracture lines via a *fracture map*. This is very useful for chipping tiles or bricks off of walls, leaving a backface that is unfracturable. This allows you to insert destruction into a game without changing gameplay (since a new path won't be opened up by allowing destruction of the whole wall).

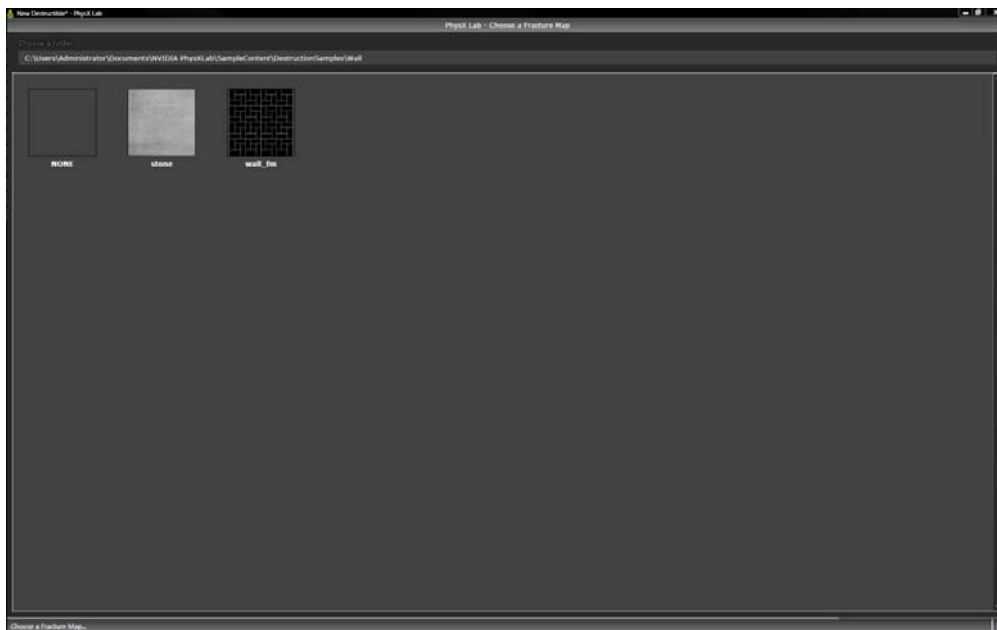
For this example, we'll start with a new project by choosing "New" under the File menu. Import the wall.fbx mesh, by choosing "Import 3D Mesh" under the File menu. This wall has its face normal along the Y axis.

You should see



To select cut-out fracturing, choose the "chipped" icon on the toolbar:

The Tools panel on the right will now show cut-out fracturing controls. The first thing you'll need to set is a *fracture map*. To do this, click on the large, black square in the Fracture Map section. After doing this, a loading pane will appear which is similar to the project loading pane:



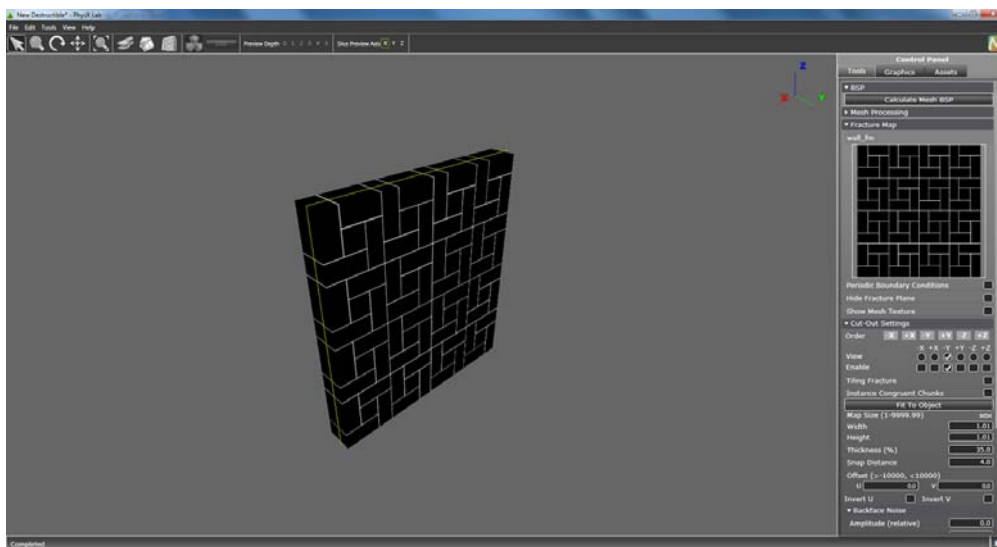
The main panel shows all of the bitmaps in the current folder. To change folders, click on the folder path above the main panel and a folder browsing dialog will appear.

Note Currently, fracture maps only supports bitmap (.bmp) file, but any pixel depth is supported.

The fracture map will define the location of fracture lines using the light areas of the map contrasted against the dark.

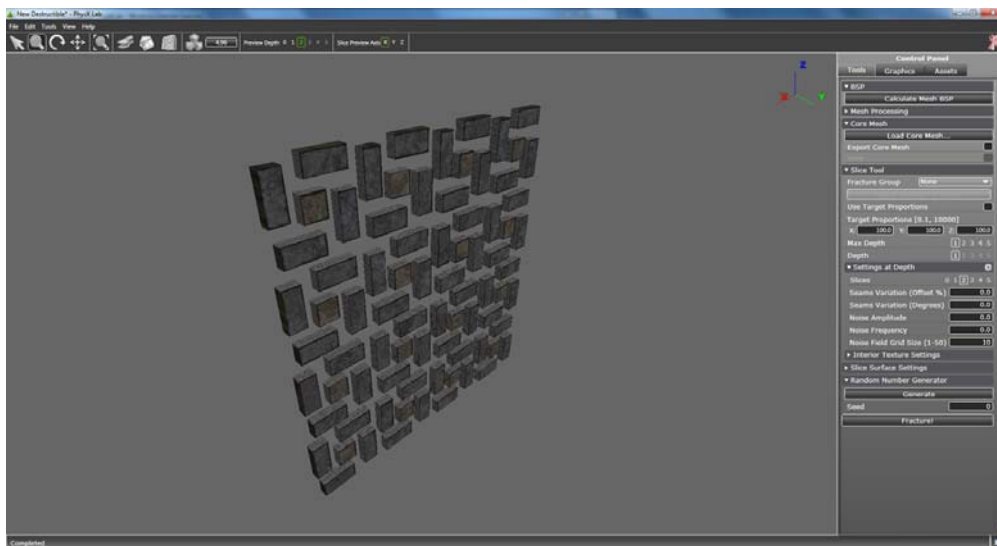
Choosing wall_fm, which was drawn to fit the wall mesh (and especially the brick markings on the surface material), you will return to the main editor with that map in the Fracture Map control.

You will also see how the map will be applied to the mesh. The default is to apply the map in the -X direction, which obviously is incorrect for this mesh. To fix this, **uncheck** the **-X** checkbox in the Cut-Out Settings section (under the fracture map), and **check** the **-Y** checkbox. Then click the radio button above the **-Y** checkbox to view how the map will be applied. You should see:



One thing you'll almost certainly want to adjust is the cut-out thickness. This is given in the Thickness control under Cut-Out Settings. This is a percentage of the mesh thickness in the cut-out direction.

For this example we'll set the thickness to 35%, and also add some backface noise. This is controlled under "Backface Noise," with parameters exactly like those used for slicing. In this example we'll set the the Amplitude to 3%. Finally, to ensure we don't slice a sliver off of each edge, we've set the **Width** (under the **Map Size** panel) to 1.01. Since Width and Height are linked (see the link icon next to the Map Size title), this also changes the height to 1.01. This will expand the map 1% in size. Finally we click "Fracture!", and have:



Notice that cut-out fracturing creates chunks down to depth 2. It is worth exploring the chunks at other depths at this point, using the **Depth (Preview)** control. At depth 0, you have (as always) the unfractured mesh. At depth 1, you have the wall with one slice, to separate the face from the rest of the wall. At depth 2, the face is cut into the pattern given by the fracture map. Only the face has children at depth 2. The remainder of the wall is *not* fractured any further, so it has no children at depth 2.

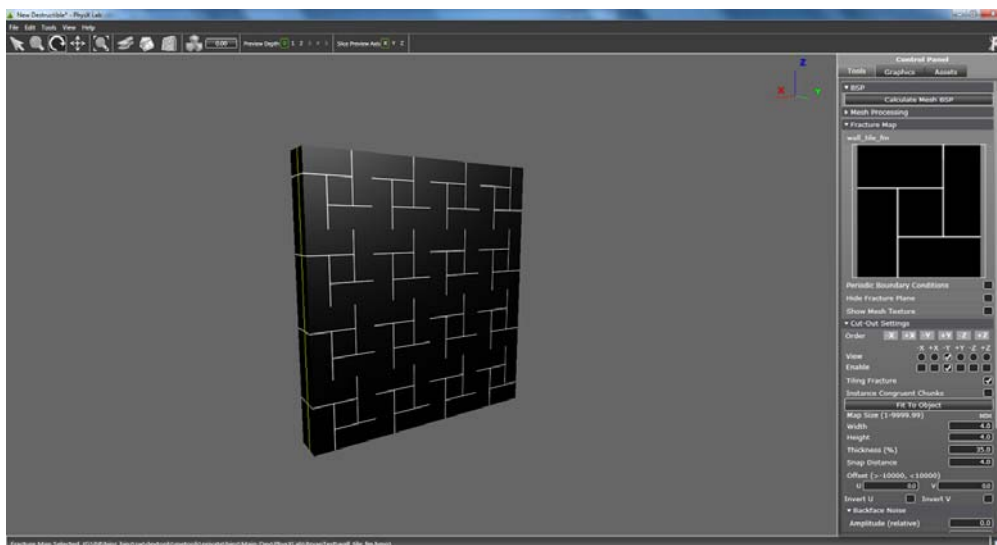
A couple of notes:

Note If you select more than one direction in which to apply the fracture map, this will generate faces at depth 1 for each cutout direction.

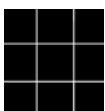
Note There is an option to break down the cut-out pieces further, using slice mode fracturing (Apply Slice to Cutout) or Voronoi mode fracturing (Apply Voronoi to Cutout). This will be described in detail later.

Tiling the Fracture Map

Sometimes you may have a simple fracture map but want one composed by repeating the simple one. Now you needn't to do the composing job. Checking the **Tiling Fracture** checkbox will allow you to tile a fracture map across the mesh surface. Under this mode, **Width** and **Height** will stand for the repetition times. For example, we can specify the numbers to be 4.0, then the original map will be tiled four times to cover the mesh both horizontally and vertically. See following picture where we have used only a subset of the large brick fracture map, wall_tile_fm.bmp, and allowed it to repeat across the mesh.



It's also possible to specify periodic boundary conditions when tiling. The fracture map wall_tile_periodic.bmp generates the same fracture map as wall_tile.bmp if **Periodic Boundary Conditions** are chosen. However, two sides of this periodic map have no lines along the edge. This allows the tiled regions to effectively "wrap" to the other side of the map, allowing cutout patterns where the repeating portion is not confined to a rectangle. For example, with this fracture map,



by choosing **Tiling Fracture** and setting both repeating times as 2, we can see the different cut out results with and without of periodic boundary conditions:



Before trying this destructible out in the playground, let's fix up the texture map applied to the interior triangles. It's especially obvious at depth 1 that we need to change the material or at least stretch out the texture map. This will be covered in the next section.

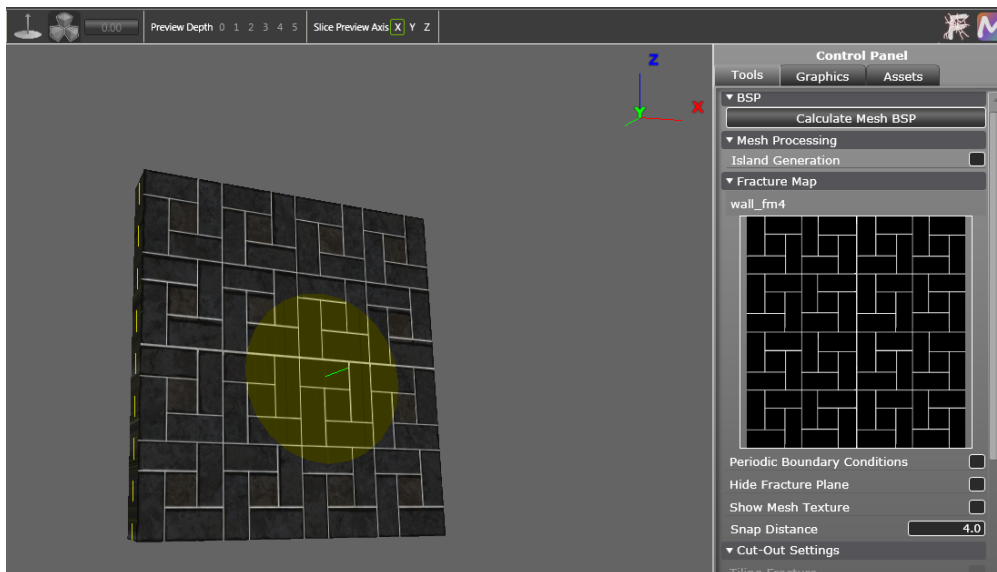
UV-Based Fracturing

This is a special cutout fracture mode. The artist may want his/her mesh to be cut out in a pattern that exactly coincides with its texture. In this kind of cases, the artist needs first to create a cutout fracture map that has the same "texture lines" as texture of the texture map. Then PhysXLab could project the fracture map onto the mesh surface because vertices of the mesh have the same texture coordinates on it as those on the texture map.

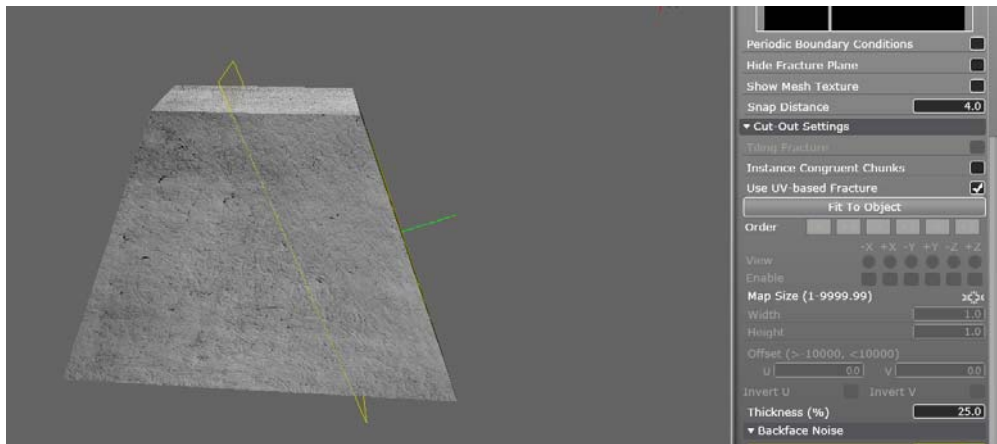
If the UV-Based mode is checked, then any face of the plane could be chosen as the cutout face. Here is the tool for picking a cutout face:



When you click on the mesh surface with this tool, the picked point in the surface and surface normal at the point is immediately displayed on top of the mesh by an arrow and a tangent plate, which would help to make your selection more accurate. The mesh face that contains the picked point and orients the same with the picked normal will be treated as the cutout face, and fracture map is projected onto the face by the way discussed above, with mesh texture visible simultaneously, to let you view how exactly the fracture lines coincides with the mesh texture. Here is an example:



You can also adjust the cutout thickness in this cutout manner, and notice that this thickness is under the meaning of the cutout direction, which is not necessary an axis-aligned one. See the example bellow, which has a thickness of 25% in the picked direction:



When you feel all settings are OK, just press "Fracture!" button, everything happening next is like that under the "normal" cutout fracture.

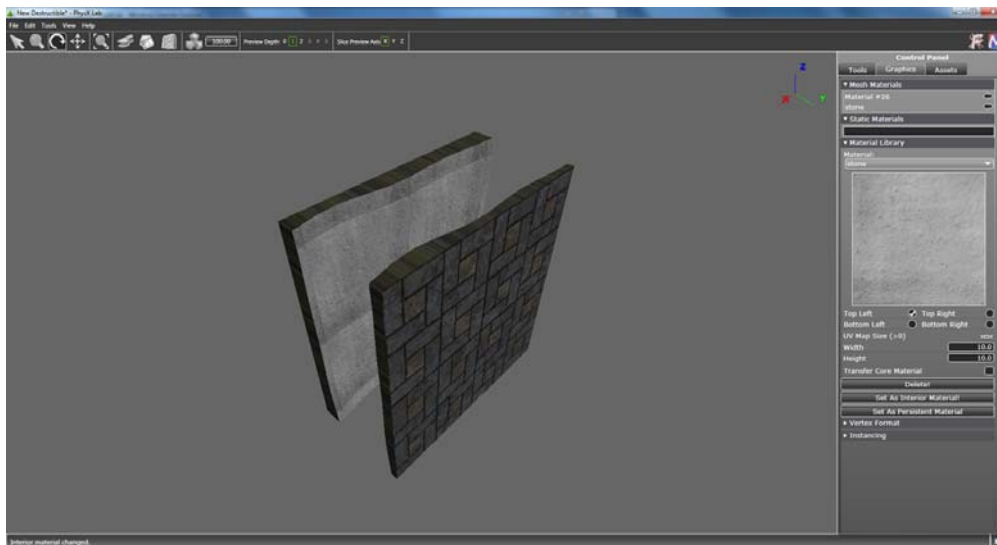
Setting the Interior Material

The faces newly-formed by splitting will have a material applied to them that is arbitrarily chosen from the mesh's current material set. For each slicing surface, texture (u,v) coordinates are projected along the surface normal, at a scale set by the user. There is currently no provision to set a (u,v) offset, or rotation for the texture coordinates. The idea is that an interior material's texture map is probably going to be a repeating, homogeneous image depicting something like cement or woodgrain, which looks acceptable with any offset or rotation.

Continuing with the wall example, select the "Materials" icon from the Control Panel in the upper-right. You will see a list of the materials currently in the mesh, as well as the materials in the tool's current "Material Library." This library is created initially from the mesh's materials, and can be saved off and re-loaded for use in other meshes if you wish. You can also add to the material library, as we will in this case to create a new material for the interior triangles.

Under the "File" menu, select "Load Texture As Material...". A standard file dialog will appear, from which we will choose "stone.bmp." After doing this, the stone texture map will appear as a new material in the Material Library.

Adjust the scale by setting the Width (under UV Map Size) to 10.00. Since the width and height are linked (note the chain icon), the Height will change to 10.00 as well. After you have done this, click "Set As Interior Material." After some processing, you should see



The stone.bmp texture map doesn't really tile, but you get the idea.

Specifying some Persistent Materials

When importing a new mesh, the previously loaded materials will be removed by default. If you want a material to be a **Global** one, that's to say, each time a mesh is imported, the material will still stay there, you can first choose the material from the droplist, and hit the button named as "Set As Persistent Material". Then the chosen material will stay persistent until you delete it.

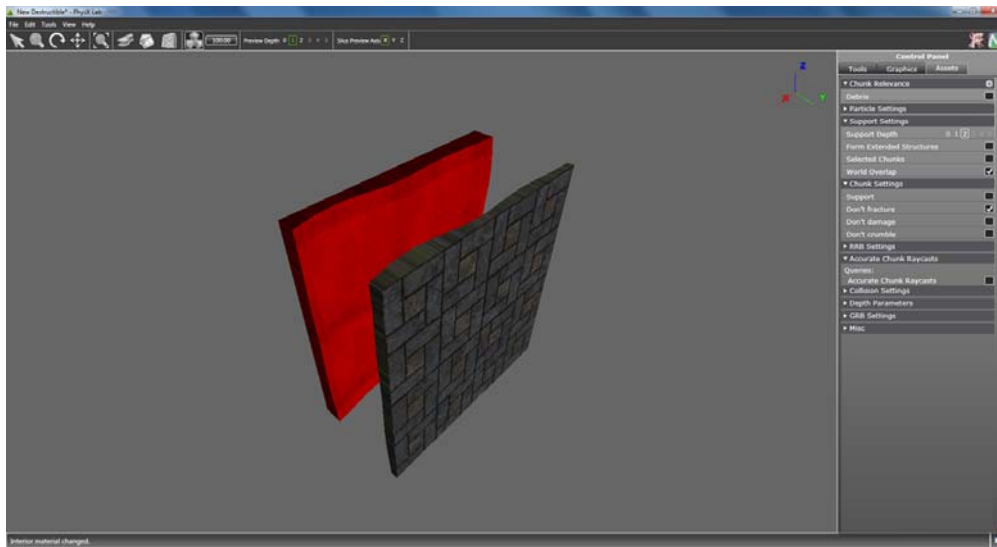
Now we have a few asset settings to change before trying the wall out in the Playground.

Asset Settings for Cut-out Fracturing

Just as with the first (block) example, click on the "Assets" icon in the Control Panel. This time, Support Depth = 2 is best, in order to keep the tiles "stuck" to the wall unless fractured loose. Again, select "World Overlap" for environmental support.

So far we've done nothing new, but now we will change two things, that you'll usually find useful for cut-out fracturing:

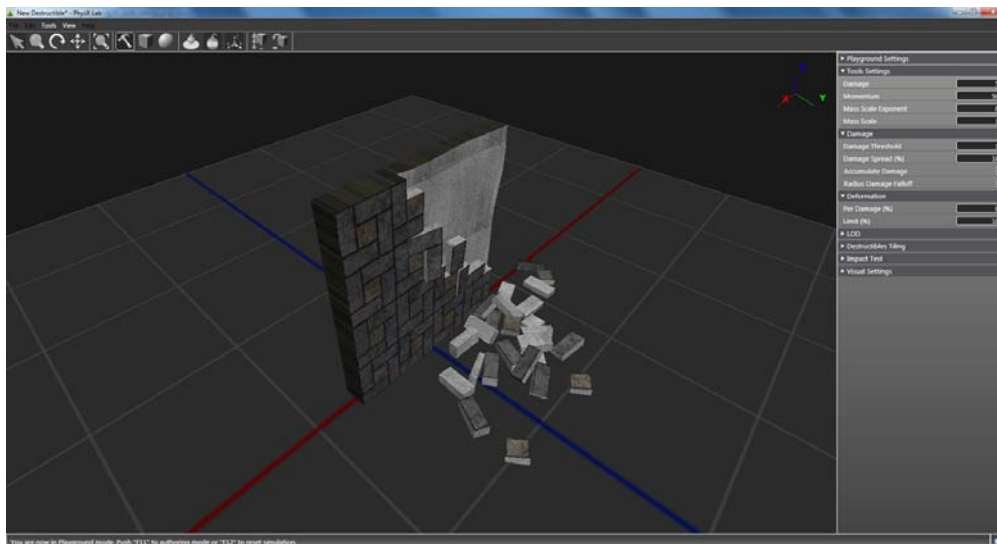
Select the "Tools" icon in the Control Panel and change the Depth (Preview) to 1. Now return to the "Assets" panel and select the back of the wall (not the sliced-off face) by clicking on it in the view panel. It will be highlighted in red. Then, select "Don't fracture" under "Chunk Settings." See the image below.



This keeps the back part of the wall from being fractured loose. This way the wall will remain a static part of the environment.

Now, select the front face by clicking on it in the view panel, and this time choose "Don't damage." This is a special setting which prevents the front face from coming off all in one piece. Any damage applied will always be propagated down to its child chunks, freeing them individually instead.

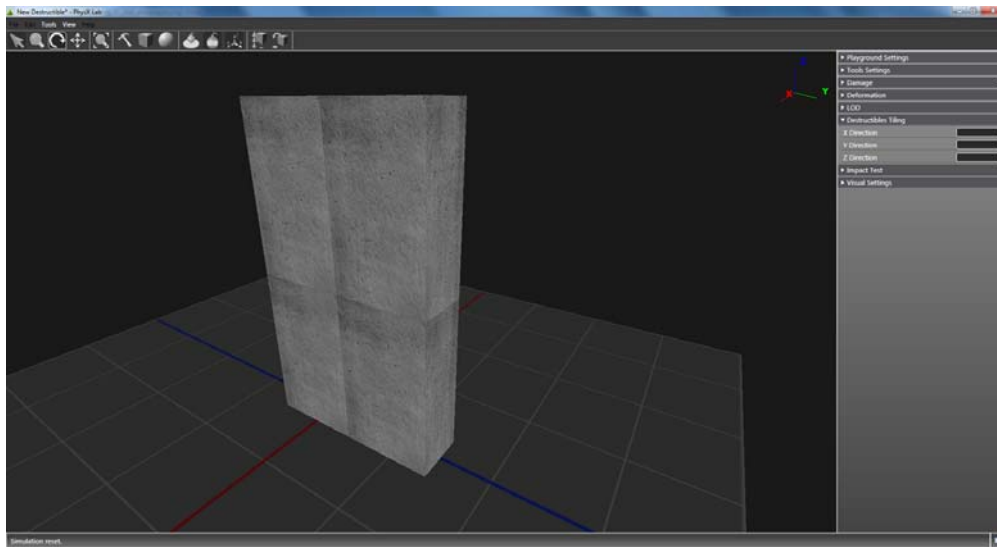
Hitting F12 to enter Playground mode, and applying damage should give a result like this:



One fun thing to try is to apply damage to all of the chunks at the base of the wall. Then, since those are the only chunks touching the "world" (the ground plane), the rest of the tiles will lose environmental support and break free together. This conglomerate can be fractured further by applying damage.

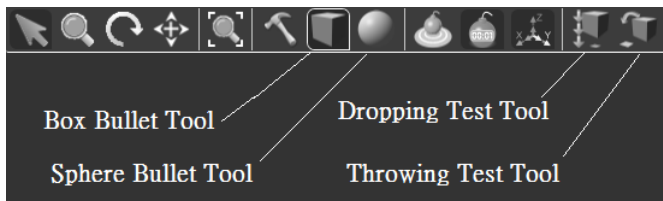
Multiple Actors in Playground

In playground settings, there is a **Destructibles Tiling** option, which allows a user to create multiple destructible actors tiled on the playground. For example, you can specify some integer numbers, say, 1, 2 and 2, for the 3 axes, then hit F12, and you'll see totally $1 \times 2 \times 2 = 4$ actors lying there.



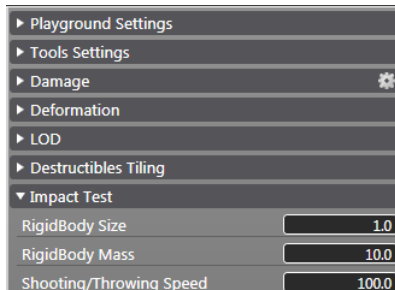
Impact Test

There are several ways to implement an impact test, which are all straightforward by having a look at the toolbar. But before a test, the **Take Impact Damage** checkbox should be checked first of all, for any depth at which you want to see impact damage taken. Then you can go to playground with a destructible actor.



Shooting a Box/Sphere

Just choose the Box or Sphere Bullet tool, click left mouse button, then a box/sphere will be shot out onto the destructible, generating an impact. When such tools are chosen, controls under **Impact Test** expander will be enabled. You can then specify size, mass and speed for your bullets. See following.



Dropping

Choose the Dropping tool shown above, move mouse cursor to any location in the playground pane. When pushing left button down, you'll see the destructible is hanging in the air, just at the height your mouse location specifies. Let the button go up. The destructible will then drop to ground and get damaged.

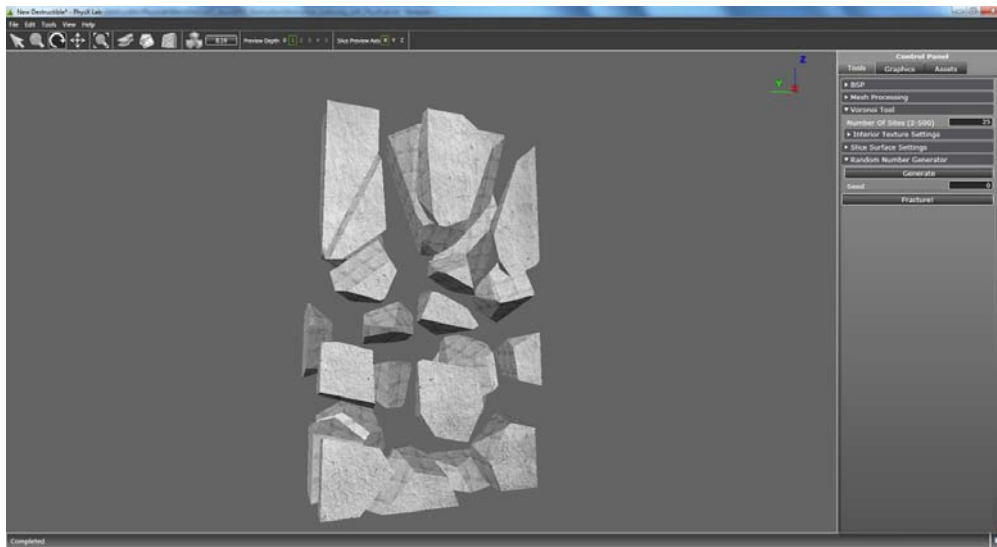
Throwing

Just like the Dropping tool, when Throwing tool is chosen, the location where left button is clicked will be the initial location for the to-be thrown out destructible. Its flying speed could also be specified in the **Shooting / Throwing Speed** control under the **Impact Test** expander.

Voronoi Fracturing

The third fracture mode is Voronoi. In this manner the user should provide a number specifying how many parts the mesh will be divided into. A 3-D Voronoi partition is calculated then, and the partition is applied to the mesh, thus dividing it into several polyhedrons. The range for the number is 2 to 1000, while the default value is 10.

Here is an example of the Voronoi fracture result with the default number 25:

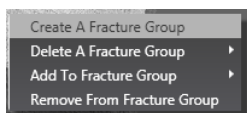


Fracture by Group

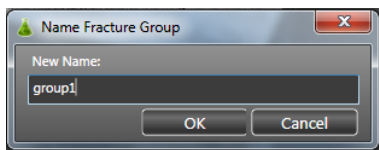
By defining and using fracture groups, you will be able to fracture any selected, and grouped chunk, while leaving all other chunks alone. You can select the Create Fracture Group option from the right-click menu, then a dialog box will pop up, waiting for you to enter a name for the newly created group. See the following two images.

You can use fracture groups either in slice mode or in Voronoi mode.

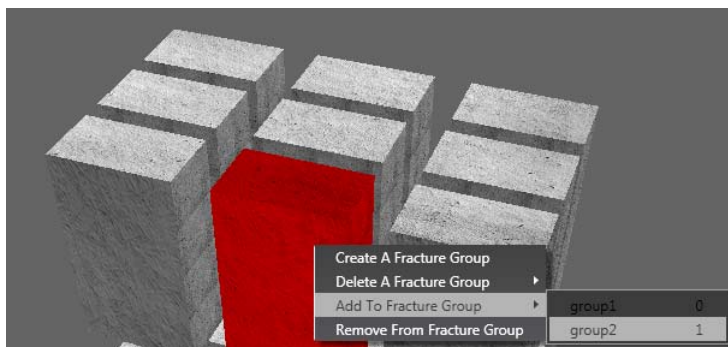
The menu items



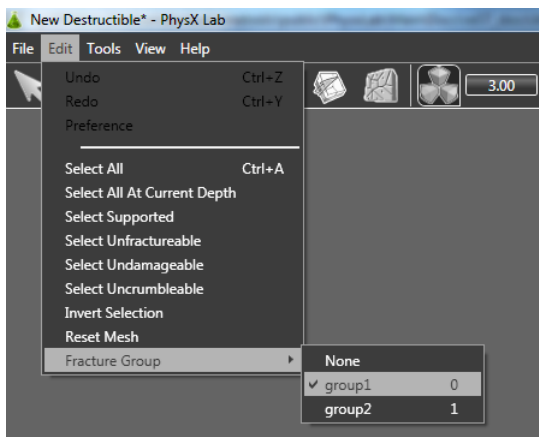
The dialog box



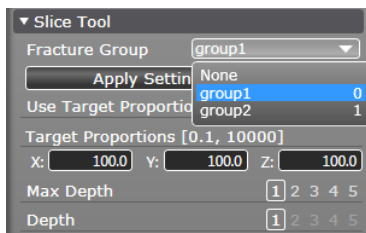
You can select any chunks and add them to a Fracture Group by option from the right-click menu(see example), and can also remove any selected chunks from their belonging groups.



A Fracture Group's depth is determined immediately the first chunk that is added to it, and after that chunks at different depths will not be able to be added to this group, unless the group becomes empty again. You can choose a chunk already belonging to one group and assign it to another, by doing this the chunk will be automatically removed from the previous group. When a group is fractured, you can select one of its children and continue to create groups with child chunks. But the deeper groups are not necessarily considered as children of the group at above depth, because deeper groups can also include chunks from different parents. We may say now only that the deeper groups are related to the above one. When we fracture a group, the related deeper groups will all be deleted. This is because the deeper groups are no longer well defined when related groups are re-fractured. You can choose a group to fracture either by selecting menu item Edit->Fracture Group-> Group Name(see image below),



or by selecting from the dropdown list shown below



Settings will be automatically saved to the group when it's created. When a group is selected from the menu, any changes to Slice/Voronoi settings will be immediately updated into this group.

By choosing the group named as "None", you will go back to the normal slicing/Voronoi mode.

Detailed User Interface Reference

Once a mesh is loaded, it will be displayed in the main view panel with three colored axes. X, Y, and Z are colored red, green, and blue, respectively.

Main Display Mouse Input

As already described in *Manipulating the View*, the manipulation mode is selected using the icon in the lower-right corner of the main panel. Clicking on it will cause all modes to be shown as icons



The current selection is highlighted, with all modes shown surrounding it. In the image above, the orbit tool is highlighted. Notice the orbit tool is also shown in the selections surrounding it.

Manipulation Modes

Mode	Description
Zoom (magnifying lens)	Left-click and drag will move the mesh towards or away from the camera.
Pan (arrows and hand)	Left-click and drag will translate the mesh in a direction perpendicular to the line of sight.
Orbit (arrows on circles)	Left-click and drag will rotate the mesh.
Explode Tool (expanding shapes)	Left-click and drag will change the distance between chunks at the currently displayed hierarchy depth. Clicking on a chunk with the center crosshair will toggle selection of the chunk. (Selected chunks are highlighted in red.)
Fit to view (box)	Automatically pans and zooms to fit the mesh within the viewport.

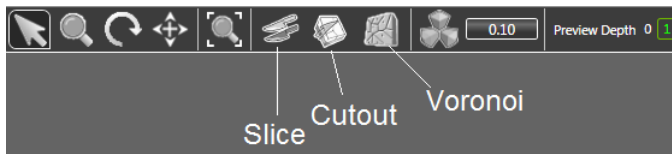
Selection

You may also use the mouse to select chunks. Selected chunks are highlighted in red.

Action	Description
Left-click on a chunk	Selects that chunk, and deselects all others.
Ctrl + left-click on a chunk	Selects a chunk, keeping any previously selected chunks selected.
Left-click not on a chunk	Deselect all chunks.

Fracture Modes

You may also select the fracture mode by clicking on the icon in the lower-left corner of the main panel



Mode	Description
Slicing (knives)	Selects slice fracturing. Slicing parameters will appear in the tool panel on the right.
Cutout (block with chipped face)	Selects cut-out fracturing. Cut-out parameters will appear in the tool panel on the right.
Voronoi (a Voronoi partition)	Selects Voronoi fracturing. Voronoi parameters will appear in the tool panel on the right.

Action Buttons

Two icons duplicate functionality in the control panels (see Control Panel, below).



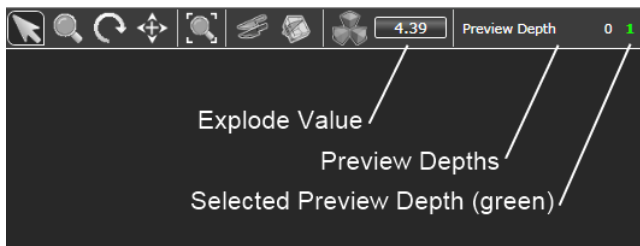
Fracture: This duplicates the **FRACTURE!** button in the Slice and Cutout mode tools panels (see below).



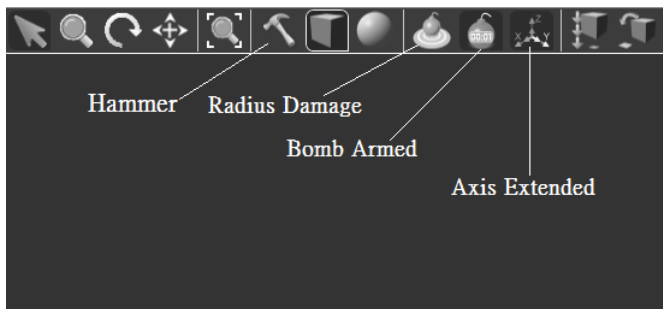
Interior Material: This duplicates the **Set As Interior Material!** button in the Materials tool panel (see below).

Other ToolBar Items

Toolbar in authoring mode:



Toolbar in simulation mode:



Item	Description
Explode Value	the amount to push the rendered chunks apart, as a percentage of a maximum expansion.
Preview Depths	the depth of the hierarchy to render. Only one level of the hierarchy is displayed at a time, so that the displayed chunks should (ideally) fit together perfectly to look like the unfractured mesh, when Explode (see above) is set to 0.
Hammer	the hitting function to fracture the mesh in simulation mode

Menus

File

Item	Description
New	Starts a new project. A mesh import dialog will pop up to begin the new project.
Open...	Currently the same as New.
Save	Save the current project. The project browser will appear, allowing you to name the current project if it hasn't yet been saved.
Save As...	Save the current project with a new name. The project browser will appear, allowing you to rename the current project.
Load Texture As Material...	Loads a texture map and turns it into a material. The material is saved in the tool's material library, and can be used in the mesh you are editing.
Create Material Library...	Stores the current contents of PhysXLab's material library into a .aml (Apex Material Library) file.
Load Material Library...	Loads a set of materials, to be merged into the current material library. Materials may come from .aml files (saved from PhysXLab), from Alias .fbx files, or from Autodesk .mtl files. If a conflict exists between material names in the loaded file and those in the current material library, a dialog will appear allowing the user to resolve the conflict.
Save Preset...	Saves the current fracturing parameters into a preset file. These may be reloaded within a different project, allowing fracturing parameters to be kept as a "library" of destructible behaviors.
Load Preset...	Load a fracture parameters preset file into the current project.
Save Fracture Data...	Saves the current mesh, in whatever fractured state it's in, in an editing format used only by PhysXLab.
Load Fracture Data...	Loads fractured mesh data saved using Save Fracture Data.
Import...	Imports a mesh from various formats. Currently two types of import are available: APEX Destructible Asset and APEX Render Mesh. The latter only contains the render data, while the former is a complete asset necessary for APEX destruction. In either case, you may import in one of the standard APEX stream formats: (a) .apx (APEX xml), an ascii format, or (b) .apb (APEX binary).

Export...	Exports the current data to various formats. Currently the same two types as above are available. In either case, you may export in one of the same two standard APEX stream formats as above.
-----------	--

Edit

Item	Description
Select All	Selects all chunks in the hierarchy.
Select Supported	Selects all and only those chunks which have been marked "Support"
Select Unfractureable	Selects all and only those chunks which have been marked "Don't fracture"
Select Undamageable	Selects all and only those chunks which have been marked "Don't damage"
Select Uncrumbleable	Selects all and only those chunks which have been marked "Don't crumble"
Select Invert Selection	Selects all chunks currently unselected, and deselects chunks currently selected.
Reset Mesh	Returns the mesh to its unfractured state, but does not erase any of your current fracture settings.
Fracture Group	Adds or removes fracture group and do slicing fracture by group. See SI- ice by Fracture Group for more details.

Tools

Most of these items are redundant ways of selecting the manipulation or fracture modes (see *Main Display Mouse Input*, above. For completeness, they are listed here:

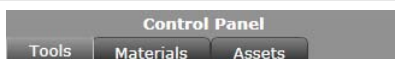
Item	Description
Orbit	Selects the orbit manipulation mode, allowing the user to rotate the mesh.
Pan	Selects the pan manipulation mode, to translate the mesh perpendicular to the line of sight.
Zoom	Selects the zoom manipulation mode, to move the mesh closer and farther from the camera.
Zoom to Selected	Automatically pans and zooms to fit the mesh within the viewport.
Cancel Fracture	Cancels an ongoing fracturing process, useful in very long fracturing process.
Options	Sets the paths to look for textures and how to handle transforms in FBX file.
Explode	Selects the explode mode, allowing the user to separate or coalesce chunks. The preview depth and explode amount will be shown on the toolbar.
Slice	Selects the slice fracture mode. In the tools panel, slice parameters will be shown.
Cutout	Selects the cut-out fracture mode. In the tools panel, cut-out parameters will be shown.
Voronoi	Selects the Voronoi fracture mode. In the tools panel, Voronoi parameters will be shown.
Connect PVD	Connect to the PhysX Visual Debugger. (See the PhysX documentation.)
Playground Mode	Switch to the runtime preview (playground). F11 returns to the editor.

Views

Item	Description
X UP	Selects the X axis as the "up" axis. This will be used in the playground as well.
Y UP	Selects the Y axis as the "up" axis. This will be used in the playground as well.
Z UP	Selects the Z axis as the "up" axis. This will be used in the playground as well.
Shading On/Off	Toggles the shading mode. (Textured/flat.)
Wireframe On/Off	Toggles wireframe rendering.
Show Collision	Toggles display of the chunk collision volumes (in red wireframe).
Show Bounds	Toggles display of the chunk axis-aligned bounding boxes (in white wireframe).
Show Tangent Basis	Displays the vertex normal, tangent, and binormal (if they exist) for each vertex. These vectors are drawn as small line segments originating from the vertex. Tangents are drawn in red, binormals in green, and normals in blue. Note, it is common to have no binormals, unless the imported mesh contained binormal information. In this case you will only see red and blue lines.
Show MeshBSP Inside Regions	Once a mesh BSP has been created (See Tools Panel, below), this toggles the display of the BSP regions which are considered to be "inside."
Show MeshBSP Outside Regions	Once a mesh BSP has been created (See Tools Panel, below), this toggles the display of the BSP regions which are considered to be "outside."
Highlight Instanced Origins	0-offset instance chunks will be highlighted in yellow if there are any chunks instanced after fracturing. See Instance All Chunks and Instance Congruent Chunks below.
Highlight Instanced Chunks	Non 0-offset instance chunks will be highlighted in blue if there are any. This option together with the above one could be available at the same time. When any of the two is checked, chunk selecting is disabled. See Instance All Chunks and Instance Congruent Chunks below.
Display Typical Slice Surface	Shows the curved slice face while making the mesh transparent. See Previewing Slice Surface Preview for more details.
Show Stats	Toggles an overlay which displays various mesh/destructible stats, such as number of chunks, data size, etc.
Reset Camera	Moves the Orbit/Pan/Zoom settings back to their default initial states.
Show Log/Output	Toggles the display of the output window at the bottom of the application. It shows all info/warnings/errors of the application.

Control Panel

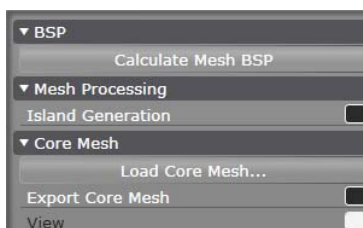
The Control Panel allows you to select between various control sets. There is a different Control Panel for the editor and for the Playground.

Editor Control Panel

Here you have the option of selecting the Tools, Materials, and Assets controls. These will now be described.

Tools (Common Items)

The Tools panel has controls which are specialized for the Fracture Mode selected (see *Main Display Mouse Input* and *Tools Menu*, above). It also has many controls which are common to all modes. The common controls are shown below.



BSP

- Calculate Mesh BSP** - Clicking this will force the calculation of a BSP for the main mesh. If this is not done, a BSP will be calculated when fracturing is performed. The reason you might want to manually force the calculation of the BSP is to ensure that it is reasonable. Using the BSP visualization options (see the **Views** menu, above), the "inside" regions should be a reasonable full-volume representation of the original mesh. If it isn't, it's possible that the BSP does not have a well-defined inside or outside. Or, the mesh may be presenting some numerical difficulty for the BSP generation algorithm. If this is the case, you can try selecting a different random seed (see **Random Number Generator**, above), and re-generating the BSP.

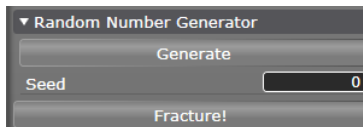
Mesh Processing / Cleanup

- Island Generation** - If this is selected, then slices that create mesh islands will be turned into separate chunks. This prevents strange-looking artifacts in which two separate-looking parts appear to be mysteriously joined.

Core Mesh

- Load Core Mesh...** - Import another mesh (.obj or .fbx) to be used as an indestructible "core" for slice-mode fracturing. When a core mesh exists, it is first subtracted from the main mesh, and then slice-mode fracturing is carried out on the remainder of the main mesh.
- Export Core Mesh** - If this is selected, the core mesh will be included in the destructible asset as a child of the unfractured mesh, and have no children (it will not fracture further). If this is not selected, the core mesh will not be part of the destructible asset.
- View** - When selected, causes the main mesh to be transparent, and renders the core mesh.

At the bottom of the Tools panel you will find these controls:



Random Number Generator

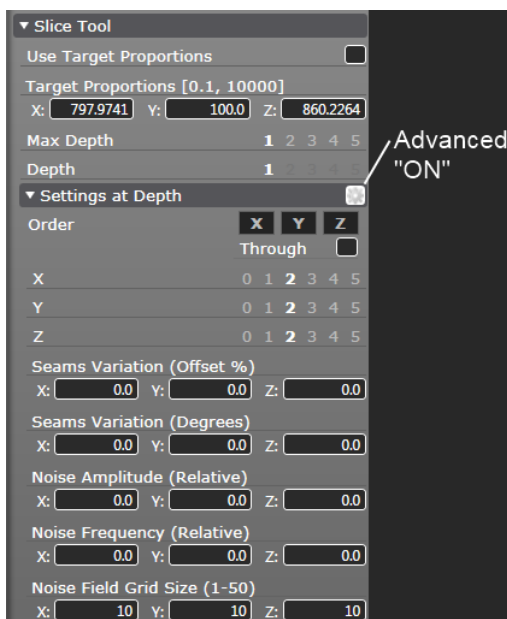
- Generate** - Generate a new seed for PhysXLab's Random Number Generator (RNG).
- Seed** - The current seed for the RNG. This may be set manually. The RNG is reset with this seed whenever fracture begins. This way the result is deterministic.

FRACTURE!

When this button is hit, the slice fracturing process will begin. If a mesh BSP has not yet been calculated, it will be now. (See **BSP**, above.)

Tools (Slice Mode)

When slice-mode fracturing is selected, these items will be displayed. Notice, there is an "advanced mode". Clicking on the "Sprocket" toggle button on the right of the "Settings at Depth" expander will toggle the mode. The "advanced mode" options are shown below.



Use Target Proportions

If this is *not* selected, the "Chunk Proportion" control (see below) will be ignored, and the number of splits along each axis, for every level of splitting, will always be given by the user selection (0-5) in the X, Y, and Z rows. If Use Target Proportions *is* selected, then the "Chunk Proportion" control gives the desired X:Y:Z size ratio for the fractured pieces. In this case, the number of splits along each axis will be calculated automatically, but always attempting to use the user's input split counts as suggestions. That is, the closest values to the user's split count values will be used which will lead to the target proportions. Since the number of splits is an integer value, the target proportions will (usually) not be exactly met at any given level of the hierarchy. But as the level gets deeper (chunks get smaller), the chunks' proportions will approach the target. Since the user's input is used as a suggestion, higher numbers will lead to more (and lower numbers to fewer) splits used by the algorithm. To see the target proportions in action, it's best to reduce the variation of the splitting surfaces ("Seams variation"). For an initial experiment, try zero variation. You'll also want to use zero Noise at first. Increase the splitting depth ("Depth" above) to 3 or 4. After fracturing, the smallest pieces should have very nearly your target proportions.

Target Proportions

If "Use Target Proportions" (see above) is selected, the desired mesh proportions for the fractured pieces.

Max Depth

How many times to recursively slice, leading to the same number of chunk hierarchical levels.

Depth

Display the slice settings at this depth. Every depth level can have different slice settings. All of the Slice controls that follow (under "Settings at Depth") depend on the Depth setting.

Order

The order in which to apply splitting along the X, Y and Z axes. For example, "XYZ" means the X splits are done first, followed by the Y splits and then Z splits. If there is no variation in the slicing surface, then the result will be independent of the order. You may change the order by dragging the **X**, **Y**, and **Z** boxes into various orders.

Through

If this is selected, "Order" (above) is ignored. For each slice along each axis, the same splitting surface is applied to all chunks (without re-randomizing the surface after the first chunk is split). This has the effect of splitting "all the way through" the mesh for each slice. In this case, the result is statistically independent of the axis order, and therefore there is only one "Through" option.

X, Y, and Z

Slices along the x-, y-, and z-axis per hierarchical pass. These are only suggestions if "Use Target Proportions" (see above) is selected.

Seams Variation (Offset %)

A random offset for the slicing surfaces, so they won't be evenly spaced. The number of slices along each axis is given at each pass. If you imagine the mesh being divided up into N equally spaced sections along an axis, then the Offset controls where the slicing plane displacements may lie within those regions. Offset = 0 means the plane displacements will always be centered in the regions. Offset = 100% means the plane displacements may lie anywhere within those regions, at a randomly chosen location. This is controlled separately for each axis.

Seams Variation (Degrees)

A random angle for the slicing surfaces, so they won't all be axis-aligned. The value you choose is the maximum angle (in degrees) that the slicing plane normals may deviate from axis-alignment, for each slice direction.

Noise Amplitude (Relative)

The amplitude of random variations applied to the splitting surfaces. The value is a percentage of the mesh size. Be careful adding noise. It makes much more complex surfaces, which tend to take much more memory to store. It also can take a long time to calculate many recursive splits with high noise along all three axes.

Noise Frequency (Relative)

The frequency of random variations applied to the splitting surfaces. The value is a percentage of the reciprocal of the grid spacing. (See Noise Field Grid Size, below.) Be careful adding noise. It makes much more complex surfaces, which tend to take much more memory to store. It also can take a long time to calculate many recursive splits with high noise along all three axes.

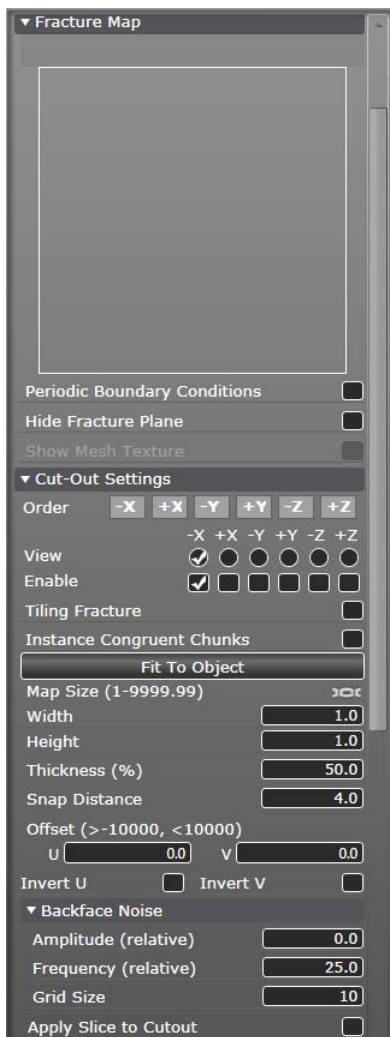
Noise Field Grid Size

When generating a noisy surface, a heightfield grid is used. This is the minimum number of grid elements across the mesh to be used, when slicing along each axis.

Tools (Cutout Mode)

Cutout mode allows an artist to define fracture lines via a *fracture map*. This is very useful for chipping tiles or bricks off of walls, leaving a backface that is unfractureable. This allows you to insert destruction into a game without changing gameplay (since a new path won't be opened up by allowing destruction of the whole wall).

When cutout-mode fracturing is selected, these items will be displayed.



Fracture Map

This displays the fracture map, which defines the location of fracture lines using the light areas of the map contrasted against the dark. To change this map (or load it initially), clicking on the map display (it will initially be black).

- **Periodic Boundary Conditions** - please reference description in the section **Tiling the Fracture Map**.
- **Hide Fracture Plane** - hide the fracture map plane, this can help visualize the preview slice surface in cutout mode.
- **Show Mesh Texture** - by checking this can temporarily hide the fracture map and at the same time make the mesh transparent, thus the mesh texture and the fracture plane covered with fracture map texture are visible simultaneously. This is useful when you want to line up the fracture map texture with the mesh texture.

Order

This controls the order in which the fracture map will be applied in the +X, -X, +Y, -Y, +Z, or -Z directions. Direction icons can be dragged into their desired order, from left to right. Only enabled directions will actually be used when fracturing.

View

The fracture map may be applied along the +X, -X, +Y, -Y, +Z, or -Z directions, in any combination. This allows you to chip away at all sides of a pillar, for example. For clarity, the map will be displayed on the mesh as applied in only one of these directions. This control allows you to choose which direction to display.

Enable

Choose which of the directions for which the fracture map is to be applied.

Tiling Fracture

Specify whether tile the fracture map across the mesh surface, or just use the original fracture map. If tiling is chosen, the **Width** and **Height** fields specify how many times to repeat the map across the mesh. If tiling is not chosen, **Width** and **Height** refer to the size of the fracture map relative to the face of the mesh.

Instance Congruent Chunks

When using tiling fracture, many chunks generated are congruent, only differing by some translation. This applies to both the render geometry and collision geometry.

With Instance Congruent Chunks selected, Congruent chunks are found and only one render and collision mesh will be generated for each congruent set. APEX will render the RenderMeshAsset associated with the destructible using instanced rendering. Note, APEX 1.1 does not instance UV offsets, only vertex position offsets. So the tiling must perfectly match a repeating pattern on the texture maps used on the face of the mesh in order for the fractured pieces to look correctly textured. (APEX 1.2 will support UV offset instancing as well, removing this restriction.)

Fit to Object

For the displayed mapping, adjusts the map proportions to fit the mesh's axis-aligned bounding box in the mapping direction.

Map Size

The chain link allows you to couple or decouple the Width and Height proportions of the map. Adjusting the width and height changes the map size as it is projected onto the mesh. The result is displayed in the main view panel.

Thickness (%)

This cutout thickness, as percentage of the mesh thickness in the cut-out direction. If there is no backface noise (see below) and the thickness is 100%, then there will be no backface slice, and the cutout will be applied all the way through the mesh.

Snap Distance

Line endpoints on the fracture map will be merged if they are further than this distance apart (in pixels). If you notice cracks between the cutouts in your fractured mesh, or other errors, adjusting this parameter can sometimes fix the problem.

Offset

Allows the user to shift the fracture map horizontally (U) and vertically (V).

Invert U / Invert V

Flips the map horizontally or vertically.

Backface Noise

These noise parameters are exactly like those used for slice-mode fracturing. See Noise Amplitude, Noise Frequency, and Noise Field Grid Size in **Tools (Slice Mode)**.

Additional Behavior

This is a dropdown list control. There are 2 additional behaviors for cutout fracturing: "Apply Slice to Cutout," and "Apply Voronoi to Cutout."

If "Apply Slice to Cutout" is chosen, slicing parameters from the Slicing fracture mode panel will be used to further fracture cut-out chunks. You must set them in the Slice mode panel and then return to the cutout mode panel to commence cut-out fracturing.

Note The "Depth" in the Slice mode tool panel will mean the *additional* hierarchy depth past that of the cutout fracture.

Note The "X" and "Y" directions in the Slice mode tool panel will refer to the directions in the cut-out plane. The "Z" direction will refer to the cut-out depth direction. This change of coordinates is done so that the slice mode parameters may apply to all cut-out directions. For example, for thin tiles, you probably want to set the number of splits per pass to 0 in the Z direction, so further slicing will not occur in the cut-out direction.

If "Apply Voronoi to Cutout" is chosen, Voronoi parameters from the Voronoi fracture mode panel will be used to further fracture cut-out chunks. You must set them in the Voronoi mode panel and then return to the cutout mode panel to commence cut-out fracturing.

If nothing is chosen, then no further behavior would be applied to the cutout fracturing.

Tools (Voronoi Mode)

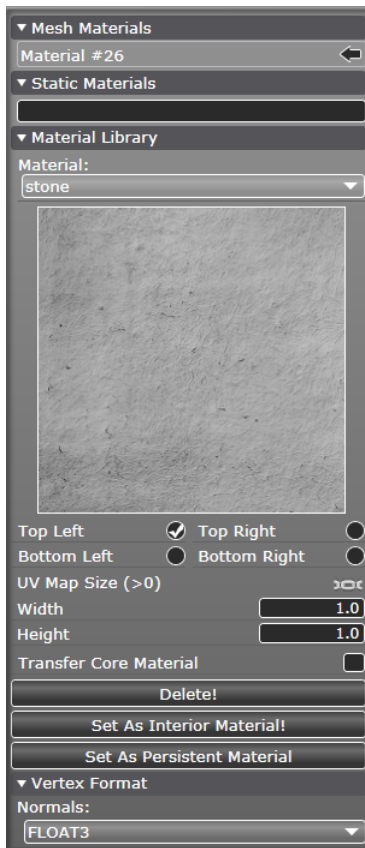
Voronoi mode allows a user to provide a number specifying how many parts the mesh will be divided into. These are defined by Voronoi cell "sites." Each site defines a convex polyhedron. These polyhedra partition the 3D space. The mesh is fractured by intersecting it with each site's polyhedron. Sites are chosen to lie within the volume of the mesh, but otherwise their placement is uniformly distributed.

Number of Sites

The user can specify the Voronoi site number here, which has a range from 2 to 1000.

Materials

This panel allows you to edit the mesh's material names, and manage PhysXLab's material library.



Mesh Materials

Displays the current materials used by the mesh. These are all simply stored as strings in the mesh. Material name references are resolved at mesh asset load during runtime, when the application can furnish a reference to an application-specific object representing the material.

For preview purposes in PhysXLab, this name will refer to a simple material library, which contains named materials that store a diffuse texture map, and some simple lighting parameters.

Double-clicking on a material name allows you to edit it. There is no restriction placed upon the material name in a mesh. However, PhysXLab will search for the renamed material in its material library and if found, also rename it in the material library, so that it will continue to be rendered.

Static Materials

Allows you to set alternate material names, to be used when the 'renderStaticChunksSeparately' bool is set to TRUE in the NxDestructibleActor descriptor parameters. When this is done, static chunks are rendered in separate draw calls. This allows applications to use a different lighting path or different shaders for faster static rendering. The alternate material names may be used to reference the static shaders, for example.

Material Library

Displays the current contents of the material library. Selecting the material from the Material drop-down will cause that material's diffuse map to be displayed in the box below. Clicking on that map display will bring up a dialog allowing you to rename the material. PhysXLab will search for the renamed material within the mesh and if found, also rename it in the mesh, so that it will continue to be rendered.

The "Top Left," "Top Right," "Bottom Left," and "Bottom Right" radio buttons allow the user to select the origin convention for the texture map.

UV Map Size

These sizes, Width and Height, can be coupled (see the chain icon). When coupled the aspect ratio of the width and height is preserved. When uncoupled (by clicking on the chain, the width and height may be adjusted separately. These values define a scale at which texture coordinates are created for interior triangles, generated by the fracture process. These values are the width (in the mesh's vertex coordinate units) spanned by the width and height of the texture map.

Transfer Core Material

If you are using a core mesh in Slice mode (see **Tools (Slice Mode)** above), and this is selected, then the material on the core mesh will be used on the surfaces of the chunks it cuts into. This will be the case whether or not the core mesh is exported (see **Export Core Mesh**). If this is not selected, the chunks will use the interior material (defined above) on the surfaces that the core mesh cuts into.

Delete!

Causes the selected material in the material library to be deleted. Useful when you want to pare down the material library to only the ones needed by the mesh, for example, before saving the material library (see **File** menu, above).

Set As Interior Material!

Causes the selected material to be used as the interior material. If the mesh is not yet fractured, and there are no material triangles, this material will be used when interior triangles are created by the fracture process. If interior triangles already exist (in a fractured mesh), then this material will be applied to those triangles, removing whatever material and (u,v) mapping previously existed for them.

Set As Persistent Material

Makes the selected material global. When new meshes are imported, the global materials will stay there. They stay persistent until user deletes them.

Vertex Format

This panel allows the user to set the data format used for various channels in the render mesh's vertex buffer. Currently only the "normals" (which include normal and tangent) channels may be set.

- Normals** - the data format for the normal and tangent channels in the vertex buffer.
 you may select "FLOAT3" for (float,float,float) format, or BYTE_SNORM3 for (byte,byte,byte) format. The latter discretizes the components into a -127 to 127 integer range, with a bias of 127.

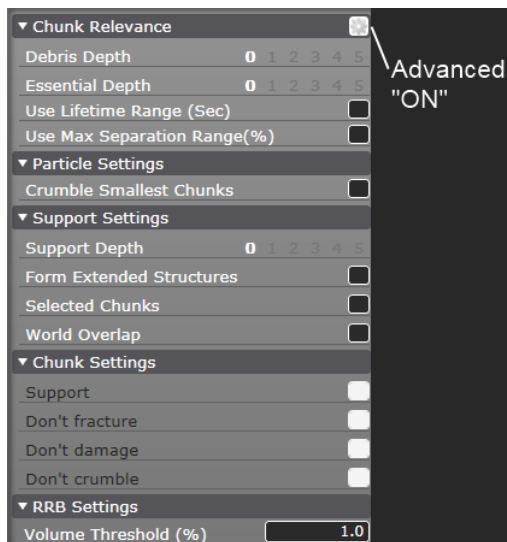
Instance All Chunks

This option allows chunks to be instanced across different assets. Note, if **Instance Congruent Chunks** is chosen, the congruent chunks already get instanced across different destructible actors as well. However, **Instance All Chunks** chosen, the remaining chunks (not part of any congruent set) will also get instanced between different destructible actors. This option is always available for any fracture mode, whereas **Instance Congruent Chunks** is only available on cutout fracturing.

Assets

This panel allows you to set various properties of the destructible asset, including some that are set on a per-chunk basis.

"advanced mode" button is toggled below in order to display all controls.



Chunk Relevance

- Debris** - if set, then Lifetime Range and Max Separation Range (see below) will be set.
- Debris Depth** - the chunk depth which will be considered "debris." All chunks at this depth (or deeper) will have debris settings applied to them. That is, they Lifetime range and Max Separation Range (see below) will apply to them.
- Lifetime Range (Sec)** - if selected, the Min And Max times (in seconds) will be used to calculate the chunk lifetimes. The actual value used is based upon an LOD setting in the APEX runtime Destructible Module. It will be within the range given. After a debris chunk has exceeded its lifetime, it is removed from the scene.
- Lifetime Range (Sec)** - if selected, the Min And Max times (in seconds) will be used to calculate the chunk lifetimes. The actual value used is based upon an LOD setting in the APEX runtime Destructible Module. It will be within the range given. After a debris chunk has exceeded its lifetime, it is removed from the scene.
- Max Separation Range (%)** - if selected, the Min and Max values will be used to calculate a maximum distance the chunk may be separated from its place of creation. The actual value used is based upon an LOD setting in the APEX runtime Destructible Module. It will be within the range given. After a debris chunk has exceeded its maximum range, it is removed from the scene.
- Essential Depth** - chunks down to this depth are considered "essential," and will not be removed by budget-based LOD by the Destruction Module. This is useful for items which are essential for gameplay.

Crumble Settings

- Crumble Smallest Chunks** - if selected, then the smallest chunks in the hierarchy may be destroyed if enough damage is applied. When this happens, a crumble particle system may be used to emulate destruction by crumbling into pieces. This crumble particle system is chosen using "Choose a .mpf file..." (see below).

- **Choose a crumble emitter .xml file...** - choose the mesh particle factory file used to generate mesh particle crumbsles. See "Crumble Smallest Chunks" (above).
- **Choose a dust emitter .xml file...** - choose the mesh particle factory file used to generate dust particles along the cracks where a chunk breaks off.

Support Settings

- **Support Depth** - a support graph will be calculated (and maintained at runtime) for chunks at this depth. This allows chunks at or below the support depth to "stick together." Any chunks below the support depth automatically become free once fracturing occurs down to a level at which they're exposed. Ideally, you'd probably want the support depth to be as deep as possible, that is the maximum chunk depth. But for deep enough hierarchies, say down to a depth of three or greater, the number of chunks becomes large enough that maintaining the corresponding support graph can become cost prohibitive. The support graph is also used for "environmental support." That is, APEX destructibles can mimic being attached to the world. When that attachment is severed, the rest of the destructible can break free. Environmental support may be chosen for various chunks in a couple of ways, see "Selected Chunks" and "World Overlap" below.
- **Form Extended Structures** - any two destructibles with this selected, when placed in contact with each other in a level, will form an "extended structure." This means that there will be one support graph common to them, which determines environmental support. This allows several destructibles to be placed together as "building blocks," and act as a single, large destructible.
- **Selected Chunks** - if this is selected, then chunks which are tagged as "Support" (see Chunk Settings, below) will be considered "environmentally supported." ** PLEASE NOTE, if both "World Overlap" and "Selected Chunks" are selected, then chunks must pass *both* criteria to be environmentally supported.**
- **World Overlap** - if this is selected, then chunks which touch world geometry will be considered "environmentally supported." ** PLEASE NOTE, if both "World Overlap" and "Selected Chunks" are selected, then chunks must pass *both* criteria to be environmentally supported.**

Behavior Groups

Behavior Groups are a new feature in APEX 1.3. Instead of having only asset-global settings such as a damage threshold, or per-chunk flags such as support, Behavior Groups offer a compromise that allows different chunks to have different settings by grouping those settings into "Behavior Groups," and letting chunks refer to those groups by index.

There is always a default behavior group, which chunks use by default. It is indexed by the number -1. If the user creates behavior groups by pressing the Add Behavior Group button, then the indexed custom groups start at index 0.

The Behavior Group has the following fields:

- **Damage Spread** - These values define the radius of damage taken by a destructible, and the amount of damage taken. For a given damage radius R (point damage has zero radius), a maximum radius is calculated using $(\text{max radius}) = (\text{Min Radius}) + (\text{Radius Multiplier}) * R$. Full damage is taken up to the minimum radius, and damage falls off to zero at the maximum radius, if the falloff exponent is greater than zero. An exponent of 1 gives linear falloff.
- **Damage Color Spread** - Similar to Damage Spread, but controls damage vertex coloring. See Damge Color Change.

- **Damage Color Change** - If any of these values is non-zero, then when damage is applied to a chunk, its vertex color is changed by the amount given, multiplied by the damage divided by the damage threshold. Values are clamped between 0 and 255. A separate per-actor color channel is instantiated for this purpose. The initial value of this color channel is all zero, unless a color channel exists in the render mesh asset. In that case, the initial value is taken from the render mesh asset.
- **Damage Threshold** - This is the damage threshold used for the chunk. It is no longer global to the destructible.
- **Impact Damage to Radius** - For impact damage, the damage taken is this value multiplied by the damage divided by the damage threshold.
- **Material Strength** - How much the destructible will push back on impacting actors. If zero, then there is no limit to this force.
- **Density** - The physical density of chunks using this behavior group.
- **User Data** - The user may input any 64-bit value to be associated with this behavior group. There are several views of this data, Byte, Word, DWord, and QWord. The smallest-numbered fields (in all views except QWord) correspond to the least significant pieces of the actual QWord stored.

Chunk Settings

Every chunk has a number of flags which may be set for it individually. To set these flags, select the chunk using mouse-click in the main view panel, or using one of the group selection options in the **Edit** menu. Once a chunk or chunks are selected, their flags may be set using the controls in this section

- **Support** - if Selected Chunks is chosen in "Support Settings" (above), then this chunk will be environmentally supported.
- **Don't fracture** - this chunk can never be broken free (only applies to chunks which are initially static, i.e. this will not apply to a dynamic destructible object).
- **Don't damage** - damage will not be applied to this chunk, but if it has children then the damage will be passed down to them. This prevents a very large chunk from coming loose in one piece. Only its children may come loose.
- **Don't crumble** - even if Crumble Smallest Chunks is selected (see "Crumble Settings," above), this chunk will not be destroyed. (Only applies to the smallest chunks, i.e. chunks with no children.)
- **Behavior Group** - Which behavior group is used by this chunk (see Behavior Groups, above).

RRB Settings

- **Vol.threshold (%)** - chunks with volume below this percentage of the whole mesh's volume will be represented by RRBs, "reduced rigid bodies," which are simulated with less fidelity but at greater speed.

▼ Accurate Chunk Raycasts

Queries:

Accurate Chunk Raycasts ☐

▼ Collision Settings

Method:

6 - DOP

Platform:

PC

Depth:

Support

▼ Depth Parameters

Depth:

0

Take Impact Damage ☐

Ignore Pose Updates ☐

Ignore Raycast Callbacks ☐

Ignore Contact Callbacks ☐

User 0 ☐

User 1 ☐

User 2 ☐

User 3 ☐

▼ Misc

Valid Bounds ☐

Min

X: -9999.9 Y: -9999.9 Z: -9999.9

Max

X: 9999.9 Y: 9999.9 Z: 9999.9

Mass Scale Exponent

Max Chunk Speed

Queries

- **Accurate Chunk Raycasts** - if this is checked, then the `NxDestructibleActor::raycast` function will perform an additional raycast once it has hit a visible chunk. All of the visible chunk's descendents will be tested for a ray hit against their collision volumes. The latest collision time among the descendents will be used to determine both the raycast's hit position and normal. However, the visible ancestor chunk will still be the one whose index is returned by the raycast function. This is useful when a mesh is "very non-convex." In this case, the parent chunk collision surface may be a long distance from the graphics mesh. If a raycast returned a hit position using this chunk's collision volume, then it could be a long distance from child collision volumes, and damage would not propagate to children. Also, the raycast result will be more consistent with the graphical mesh.

Collision Settings

- Method** - this allows you to choose one of several methods to generate a collision volume for chunks. The default is "Wrap Graphics Mesh," which attempts to create a convex hull from of the vertices in the graphics mesh. If the mesh is very complex, however, this will fail, and the 26-DOP method will be used. In any case, you may want a simpler volume than that generated by wrapping the graphics mesh. For this, various k-DOP methods are available: + 6-DOP - an axis-aligned box. + 10-DOP (X,Y,Z) - starts with a 6-DOP and slices in four diagonal directions with planes parallel to X, Y or Z. + 14-DOP (XY,YZ,ZX) - starts with a 6-DOP and slices in four diagonal directions with planes parallel to X and then to Y, or Y and then to Z, or Z and then to X. + 18-DOP - starts with a 6-DOP and slices in four diagonal directions with planes parallel to X, then to Y, then to Z. + 26-DOP - starts with an 18-DOP and slices in the eight "corner directions."
- Scales to Precook** - these are the scales (possibly nonuniform) at which the convex hulls will be cooked and serialized into an NxDestroyableAsset when exporting a .pda file (see "Exporting..." in the File menu). When instantiating an NxDestroyableActor from an NxDestroyableAsset, any (possibly nonuniform) scale may be applied to the instance. The collision hulls are similarly scaled. (See Destruction Architecture.) If the instance's scale is not one of the precook scales, then collision hulls will be cooked on the fly as they are needed in the APEX application. This leads to runtime timing spikes, as cooking can be an expensive operation. Precooking eliminates these spikes, but will create a correspondingly larger asset file to hold all of the cooked meshes at the various scales.
- Platform** - when precooking chunk collision meshes (see "Scales to Precook," above), this is the target platform used by the NxCooking library.
- Depth** - the depth to which to save overlap (adjacency data) between chunks. This speeds up the creation of support graphs at runtime, at the expense of a larger asset size.

Depth Parameters

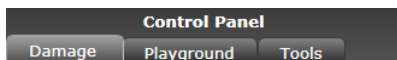
These flags apply to all chunks at the chosen depths in the hierarchy. Select the depth at which to set these flags (or "All," for all chunks).

- Take Impact Damage** - if this is set, chunks at this depth will take damage when involved in a collision. The collision force is converted into a damage amount using Impact to Damage (see the Playground Damage control panel, below).
- Ignore Pose Updates** - this flag is currently not used, and may be used as a user flag.
- Ignore Raycast Callbacks** - this flag is currently not used, and may be used as a user flag.
- Ignore Contact Callbacks** - this flag is currently not used, and may be used as a user flag.
- User 0** - a user flag
- User 1** - a user flag
- User 2** - a user flag
- User 3** - a user flag

Misc

- Valid Bounds** - if selected, then the valid bounds defined the Min and Max fields below will be used. These bounds define an axis-aligned bounding box (using Min and Max along the X, Y and Z axis). This is translated (but NOT rotated or scaled) with the destructible actor when it is created in the physics scene. (The Global Scale Factor (see above) IS applied to these bounds, however.) If a chunk leaves these bounds, it will be deleted.
- Mass Scale Exponent** - dynamic chunk islands will have their masses raised to this power. Values less than one have the effect of reducing the ratio of different masses. The closer
- Mass Scale Exponent** is to zero, the more the ratio will be "flattened." This helps PhysX converge when there is a very large number of interacting rigid bodies (such as a pile of destructible chunks). Setting this value to one will leave masses unmodified. Setting this value to zero will change all masses to 1.0.
- Max Chunk Speed** - if this is greater than 0, chunks' speeds will not be allowed to exceed this value. This value is scaled by the Global Scale Factor (see above) upon destructible asset creation.

Playground Control Panel



Here you have the option of selecting the Damage (asset damage parameters) settings, Playground settings (within the simulation environment), and Tools settings (how damage is applied).

Damage



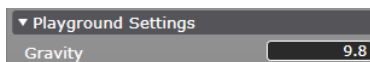
Damage

- Threshold** - how much damage needs to be applied to a chunk in order to break it off.
The scale of damage is application-defined.
- Distance Multiplier (%)** - how far damage propagates into the destructible. The damage taken is multiplied by this value, then multiplied by the size of the destructible. The result is a radius, and all chunks within that radius will receive damage. The damage amount is scaled down linearly by distance away from the damage source, so that at the full damage radius there is no damage being applied.
- Maximum** - the maximum damage that may be applied. This cap helps keep the entire destructible from being destroyed with one large application of damage, which can occur when impact damage (see "Impact to damage" below) is used. A value of 0 means there is no maximum (uncapped).
- Impact to Damage** - if this is nonzero, and "Take Impact Damage" (see the Assets Control Panel above) is selected for some level of the hierarchy, then chunks at that level will report collisions to APEX when the impact force is high enough. That impact force is multiplied by the "Impact to Damage" value and this is the damage that will be applied to the destructible, at the impact point.
- Accumulative** - if this is selected, chunks will accumulate damage applied to them.
Therefore repeated small applications of damage (each individually below the fracture threshold) will eventually fracture the chunk. Otherwise, the chunk will not fracture unless a single application of damage is large enough to do so.

Deformation

- Per Damage (%)** - if this is nonzero, then applied damage will be multiplied by this amount, and the result turned into a rotation and translation of the chunk. The chunk will not break off unless (a) the chunk's damage exceeds the "Fracture threshold" (above) or (b) the "Limit" (see below) is reached. The translation amount will be in the direction of impact, with a magnitude scaled to the chunk size (thus the "%" in this value's label). The rotation will be scaled to percent of 1 radian.
- Limit (%)** - when the deformation amount exceeds this limit (percentages in position and rotation defined in "Per Damage" above), then the chunk will fracture.

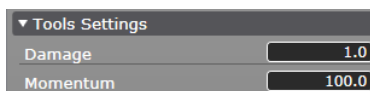
Playground



Playground Settings

- Gravity** - the downward acceleration due to gravity, as defined by the current "up" axis.

Tools



Tools Settings

- Damage** - how much damage is applied when the user clicks on the mesh with the hammer tool.
- Momentum** - how much momentum is to be transferred to the destructible when damage is applied.

Visual Settings

- Set Mesh Transparent** - make mesh transparent, only enabled when radius damage tool is active.
This is useful when you want to grab the radius tool axis which otherwise would be hidden by the opaque mesh.

How to Prep Your Mesh for Destruction

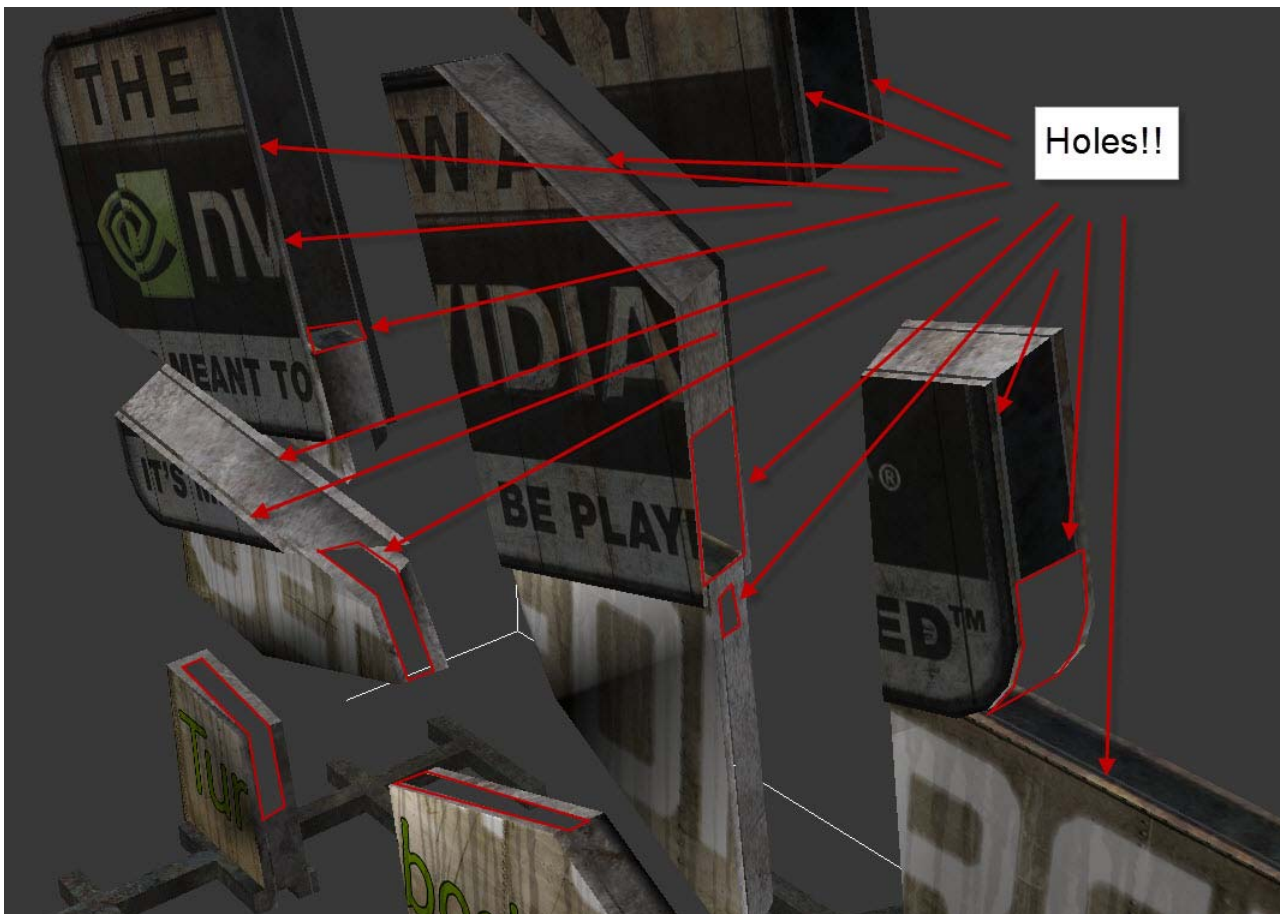
This section will go over common issues that you may encounter when trying to turn a static mesh asset into a destructible. We will go over some examples that will highlight what to do if you run into issues with your destructibles, and some of the things you can do to fix them.

The Billboard



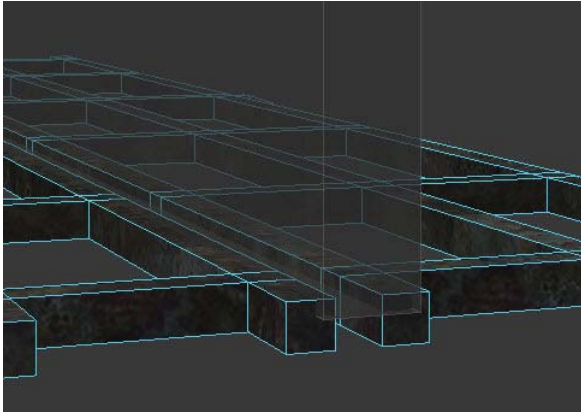
This is a simple sample asset of an old billboard. This asset is a few hundred polys with a single 2048 texture. It is a single asset made up of 5 sub-object imbedded into one another. Some of these sub-objects have open edges. For a static mesh, there is nothing wrong with this asset, but for a destructible, we have a few problems, namely the open edges, and the sub-objects being imbedded into one another.

In this asset, there are four areas that need to be fixed, three due to meshes being imbedded in each other, and the fourth due to open edges. Your first question might be why do you need to worry about this, what's the worst that could happen. Well the answer is holes, lots and lots of holes.

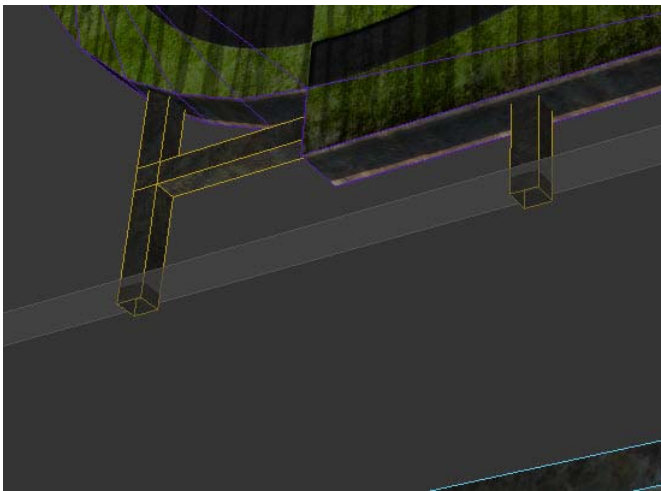


How Do You Fix This?

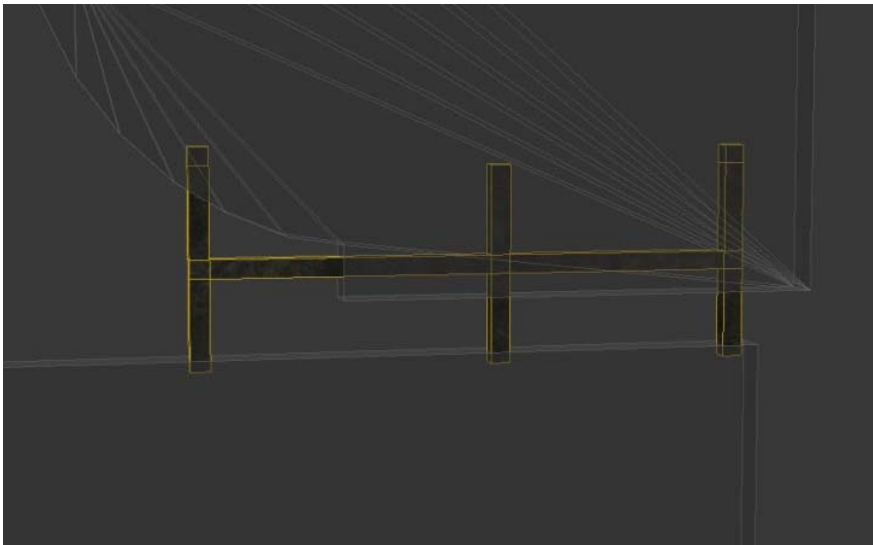
This shows the bottom of the asset, where the geforce sign, shown in X-Ray mode, meets the bottom lattice structure. You will notice how the sign is imbedded into the lattice structure. This is the easiest fix, as all you need to do is just pull the lattice out of the sign. There should be a very small gab between the surfaces to ensure you won't have any problems.



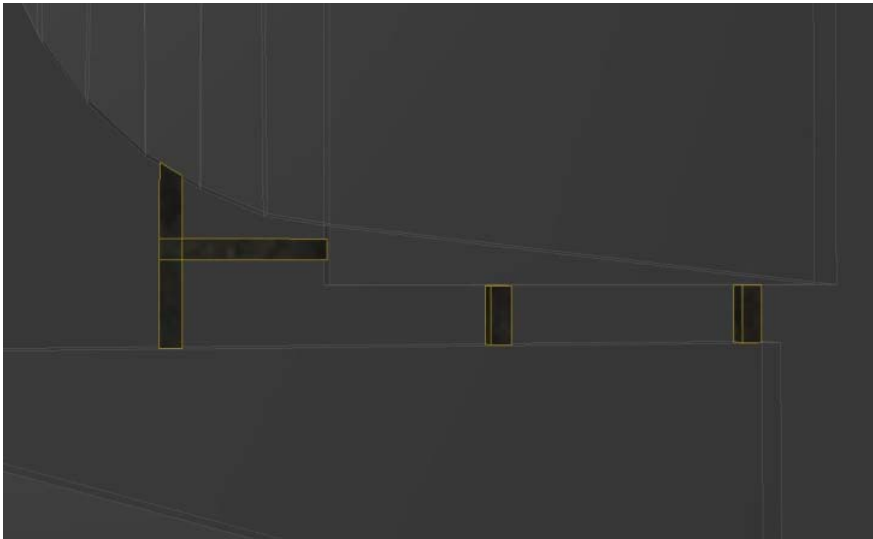
The following image shows the support structure that is holding up the nVidia logo. You can see two issues here, the first is the support structure is imbedded into the geforce sign, the second being that the support structure has open edges. There are many different techniques for closing up the holes; as long as they are closed any will work.



The next image shows the imbedding a bit better; it shows how the support structure is imbedded in both the nVidia logo and the GeForce sign. To clean this up you'll need to remove all of the support mesh that is inside the logo and sign. All you need to do is cut in some new edges, move some verts around, and delete any faces still inside the logo.



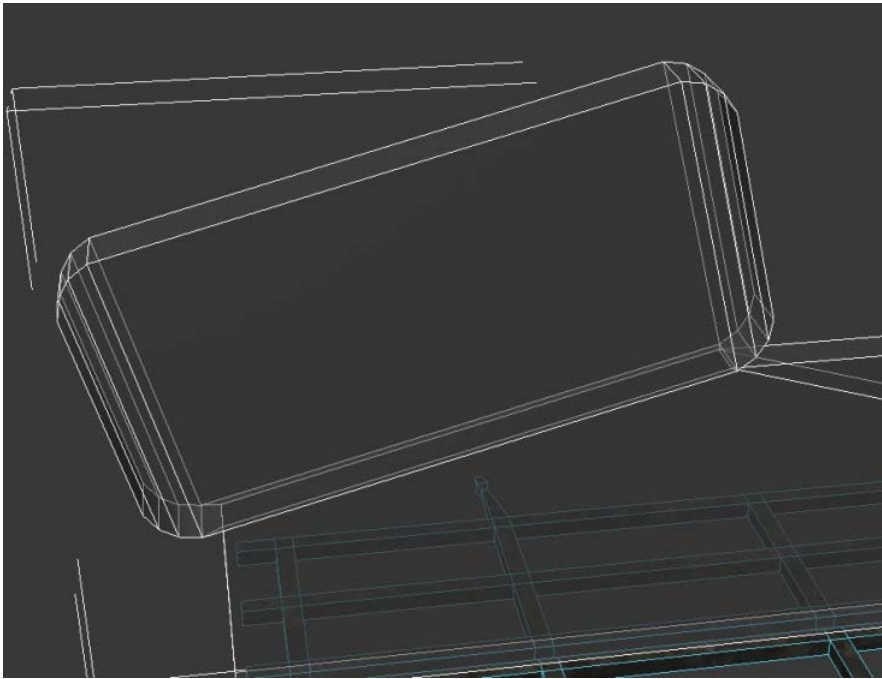
Cleaned up, it should look like the following picture.



This last piece needs the most work, as you will need to combine these two into a single sub-object. The modeling work to do this is fairly straight forwards, it's just cutting in edges, snapping verts and deleting polys.



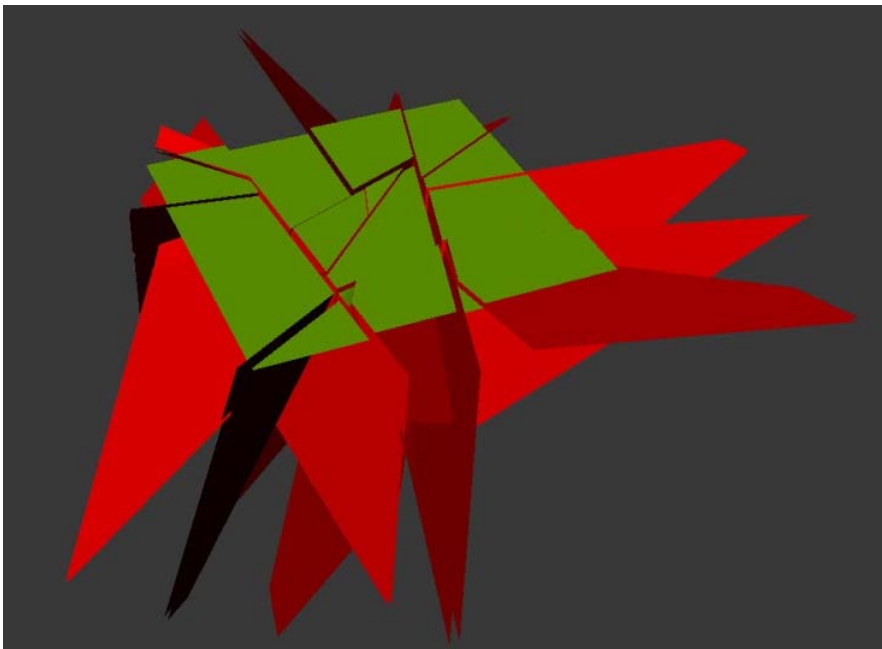
When you're done with the fixes, you should end up with something like the next image, with nVidia sign now cut into the geforce sign.



Other Common Issues

There are a few other things that a user may do when modeling that can cause holes when fractured. They are:

- **Extremely long thin, triangles.** Add an edge loop in to keep the triangles from getting too long
- **N-sided polys.** Quads and Tris are best, but it can sometimes happen that if you have an n-sided poly, you might get holes when fracturing.
- **Planar assets.** If you try and fracture something that is made up of only planes, or has a simple, 1 sided plan in it, you will not only get holes in the mesh, but extra triangles, jetting out. Destructible need to be three dimentional.



Scale of an Asset

If the scale of your asset is too big, you can find that you will get holes, this is more common in 1.2.x versions of PhysX Lab, the 1.3 version has fixed this issue. To combat this scale issue, I like to use the default max viewport grid and limit my asset to stay within the grid. While you can make your assets bigger than this, you run the risk of getting holes when fracturing.

